



МИНОБРНАУКИ РОССИИ

**Федеральное государственное бюджетное образовательное учреждение
высшего образования**

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт Информационных технологий

**Кафедра Математического обеспечения и стандартизации информационных
технологий**

Отчет по Модулю 4 – Основы языка Котлин

**по дисциплине «Проектирования и разработка мобильных приложений на
языке Котлин»**

Выполнили:

Студенты группы ИКБО-65-23

Олефилов Г.Г.

Проверил:

Дворникова Е.М.

2025 г.

Задание 1:

Параллельная загрузка и обработка нескольких источников данных

Необходимо создать консольную программу, которая делает следующее:

- Запускает три независимые «долгие» операции параллельно:
 1. Загрузка списка пользователей (delay 1800 мс) и возвращает List<String> имён
 2. Загрузка статистики продаж за день (delay 1200 мс) и возвращает Map<String, Int>
 3. Получение текущей погоды в 3 городах (delay 2500 мс) и возвращает List<String> строк вида "Москва: -3°C"
- Вывод результата только после того, как все три задачи завершатся;
- Измерить общее время выполнения;
- Добавить обработку случайного сбоя;
- Если хотя бы одна задача упала - вывести понятное сообщение, а не исключение всей программе (используй try-catch внутри async или CoroutineExceptionHandler).

Ожидаемое время выполнения примерно 2.5 сек

Примерные структуры JSON файлов

users.json

```
[
  {"id":1, "name":"Alice"},
  {"id":2, "name":"Bob"},
  {"id":3, "name":"Ivan"},
  {"id":4, "name":"Olga"}
]
```

sales.json

```
{
  "today": "2025-12-01",
  "items": [
    {"product":"Coffee", "qty":42, "revenue":1680},
    {"product":"Tea", "qty":19, "revenue":475}
  ]
}
```

weather.json

```
[
  {"city":"Moscow", "temp":-18, "condition":"snow"},
  {"city":"New York", "temp":-5, "condition":"cloudy"},
  {"city":"Tokyo", "temp":11, "condition":"rain"}
]
```

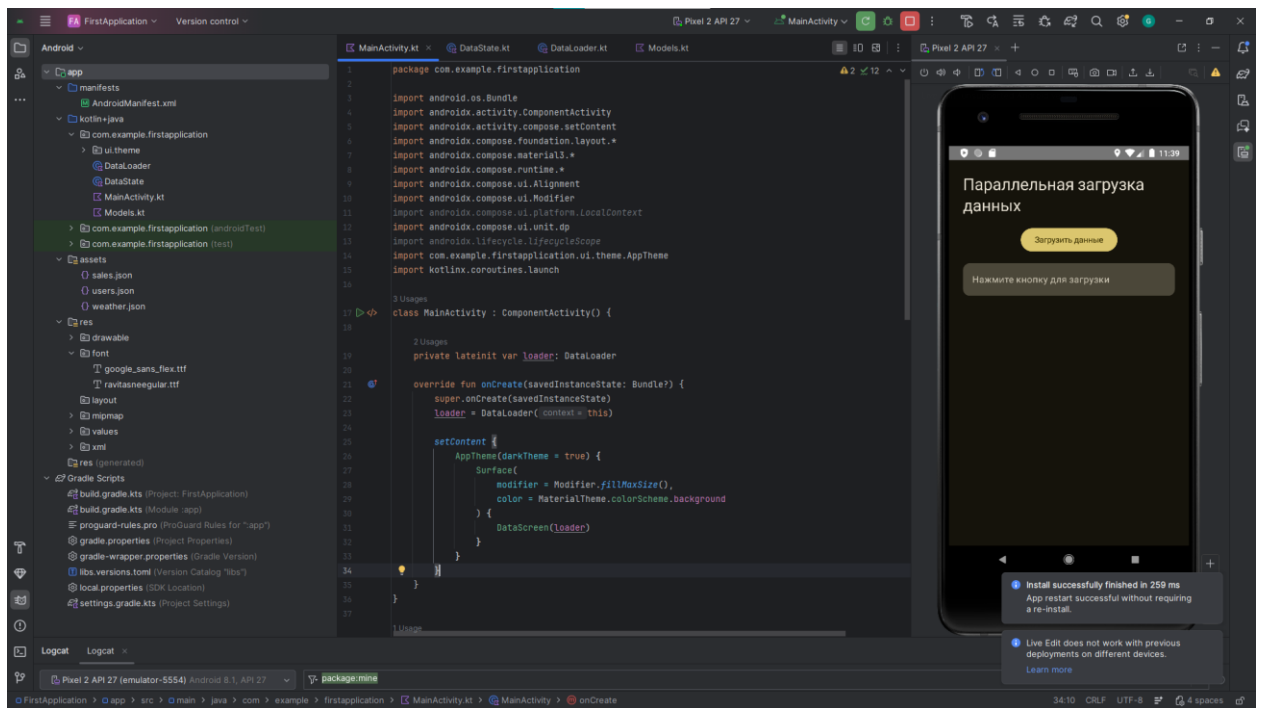


Рисунок 1 – Начальное окно приложения

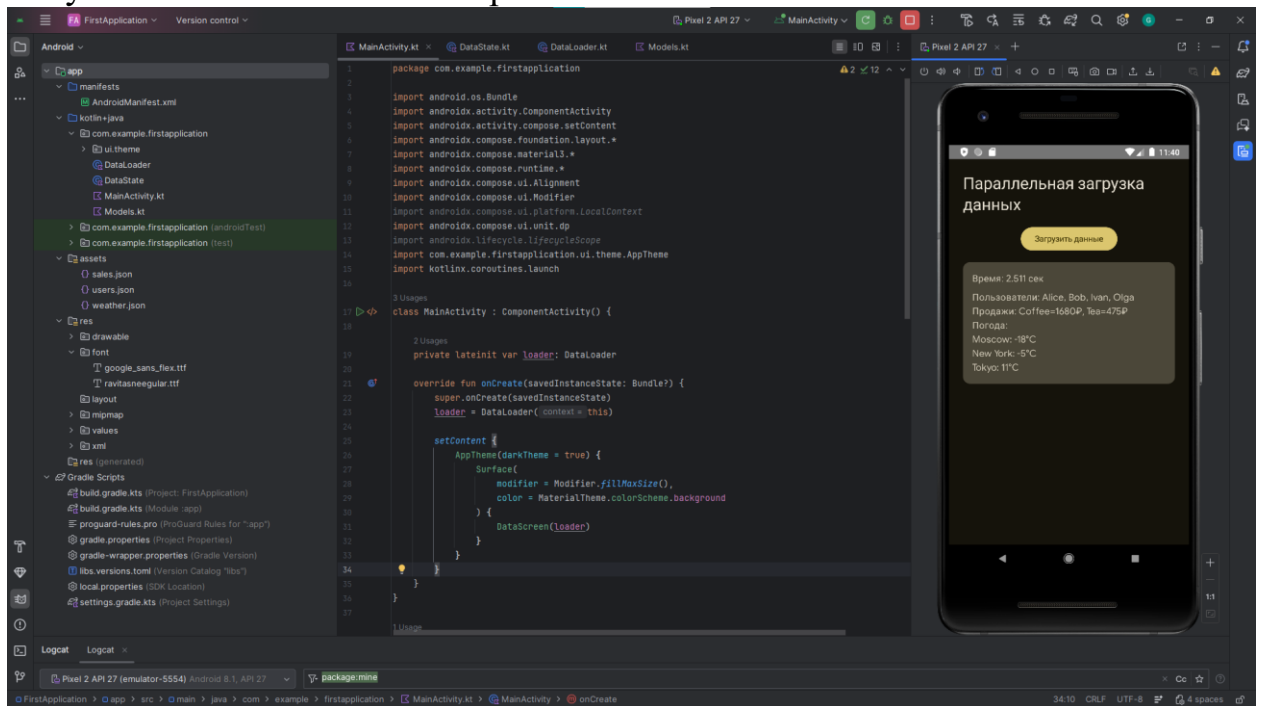


Рисунок 2 – Выгруженные данные

```
package com.example.firstapplication

import android.content.Context
import kotlinx.coroutines.*
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.serialization.json.Json
import kotlinx.serialization.decodeFromString
import java.io.IOException
```

```

class DataLoader(private val context: Context) {

    private val json = Json { ignoreUnknownKeys = true }
    val state = MutableStateFlow<DataState>(DataState.Idle)

    suspend fun loadData() {
        state.value = DataState.Loading
        val start = System.currentTimeMillis()

        val result = runCatching {
            coroutineScope {
                val users = async { loadWithDelay("users.json", 1800) {
                    decodeUsers(it) } }
                val sales = async { loadWithDelay("sales.json", 1200) {
                    decodeSales(it) } }
                val weather = async { loadWithDelay("weather.json", 2500) {
                    decodeWeather(it) } }

                buildString {
                    appendLine("Пользователи: ${users.await()}")
                    appendLine("Продажи: ${sales.await()}")
                    append("Погода:\n${weather.await()}")
                }
            }
        }

        val time = (System.currentTimeMillis() - start) / 1000.0

        state.value = result.fold(
            onSuccess = { DataState.Success(it, time) },
            onFailure = { DataState.Error(it.message ?: "Ошибка загрузки") }
        )
    }

    private suspend fun <T> loadWithDelay(
        file: String,
        delayMs: Long,
        parse: suspend (String) -> T
    ): T = withContext(Dispatchers.IO) {
        delay(delayMs)
        try {
            val text = context.assets.open(file).bufferedReader().readText()
            parse(text)
        } catch (e: Exception) {
            throw RuntimeException("Ошибка загрузки $file: ${e.message}")
        }
    }
}

```

```

    }
}

private fun decodeUsers(jsonString: String): String {
    val users = json.decodeFromString<List<User>>(jsonString)
    return users.joinToString(", ") { it.name }
}

private fun decodeSales(jsonString: String): String {
    val sales = json.decodeFromString<SalesResponse>(jsonString)
    return sales.items.joinToString(", ") { "${it.product}=${it.revenue}P" }
}

private fun decodeWeather(jsonString: String): String {
    val weather = json.decodeFromString<List<Weather>>(jsonString)
    return weather.joinToString("\n") { "${it.city}: ${it.temp}°C" }
}
}

```

Листинг 1 - DataLoader.kt

```

package com.example.firstapplication

sealed class DataState {
    object Idle : DataState()
    object Loading : DataState()
    data class Success(val data: String, val time: Double) : DataState()
    data class Error(val message: String) : DataState()
}

```

Листинг 2 - DataState.kt

```

package com.example.firstapplication

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.*
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.unit.dp
import androidx.lifecycle.lifecycleScope
import com.example.firstapplication.ui.theme.AppTheme
import kotlinx.coroutines.launch

class MainActivity : ComponentActivity() {

```

```

private lateinit var loader: DataLoader

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    loader = DataLoader(this)

    setContent {
        AppTheme(darkTheme = true) {
            Surface(
                modifier = Modifier.fillMaxSize(),
                color = MaterialTheme.colorScheme.background
            ) {
                DataScreen(loader)
            }
        }
    }
}

@Composable
fun DataScreen(loader: DataLoader) {
    val state by loader.state.collectAsState()
    val scope = rememberCoroutineScope()

    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(24.dp),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.spacedBy(16.dp)
    ) {
        Text("Параллельная загрузка данных", style =
MaterialTheme.typography.headlineMedium)

        Button(
            onClick = { scope.launch { loader.loadData() } },
            enabled = state != DataState.Loading
        ) {
            if (state == DataState.Loading) {
                Row {
                    Text("Загрузка...")
                    Spacer(modifier = Modifier.width(8.dp))
                    CircularProgressIndicator(modifier = Modifier.size(16.dp))
                }
            }
        }
    }
}

```

```

        } else {
            Text("Загрузить данные")
        }
    }

    Card(modifier = Modifier.fillMaxWidth()) {
        Column(modifier = Modifier.padding(16.dp)) {
            when (val currentState = state) {
                is DataState.Success -> {
                    Text("Время: ${currentState.time} сек")
                    Spacer(modifier = Modifier.height(8.dp))
                    Text(currentState.data)
                }
                is DataState.Error -> {
                    Text("✖ ${currentState.message}", color =
MaterialTheme.colorScheme.error)
                }
                DataState.Loading -> Text("Загрузка данных...")
                DataState.Idle -> Text("Нажмите кнопку для загрузки")
            }
        }
    }
}
}
}
}

```

Листинг 3 – MainActivity.kt

```

package com.example.firstapplication

import kotlinx.serialization.Serializable

@Serializable
public data class User (
    val id: Int,
    val name: String
)

@Serializable
public data class SalesItem(
    val product: String,
    val qty: Int,
    val revenue: Int
)

@Serializable
public data class Weather(

```

```

    val city: String,
    val temp: Int,
    val condition: String
)

@Serializable
data class SalesResponse(
    val today: String,
    val items: List<SalesItem>
)

```

Листинг 4 – Models.kt

Задание 2: Поиск дубликатов в больших файлах с timeout и отменой

Необходимо создать консольную программу, которая делает следующее:

- Принимаем путь к директории (можно захардкодить или через аргументы);
- Находим все json-файлы в директории и поддиректориях;
- Для каждого файла вычисляем SHA-256 содержимого (suspend функция);
- Запускаем поиск параллельно;
- Ищем файлы-дубликаты по SHA-256;
- У всей операции есть общий таймаут n секунд (с withTimeoutOrNull);
- Если время вышло - отменяем все корутины и выводим «Поиск прерван по таймауту»;
- Выводим группы дубликатов (если нашлись).

Для вычисления SHA-256 можно использовать: **java.security.MessageDigest**

В корне проэкт создаю Json-файлы:

```

echo '{"name": "test1", "value": 123}' > file1.json
echo '{"name": "test2", "value": 456}' > file2.json
echo '{"name": "test1", "value": 123}' > file3.json # дубликат file1
echo '{"name": "test3", "value": 789}' > file4.json

```



```
echo '{"name": "test1", "value": 123}' > file5.json # дубликат file1
```

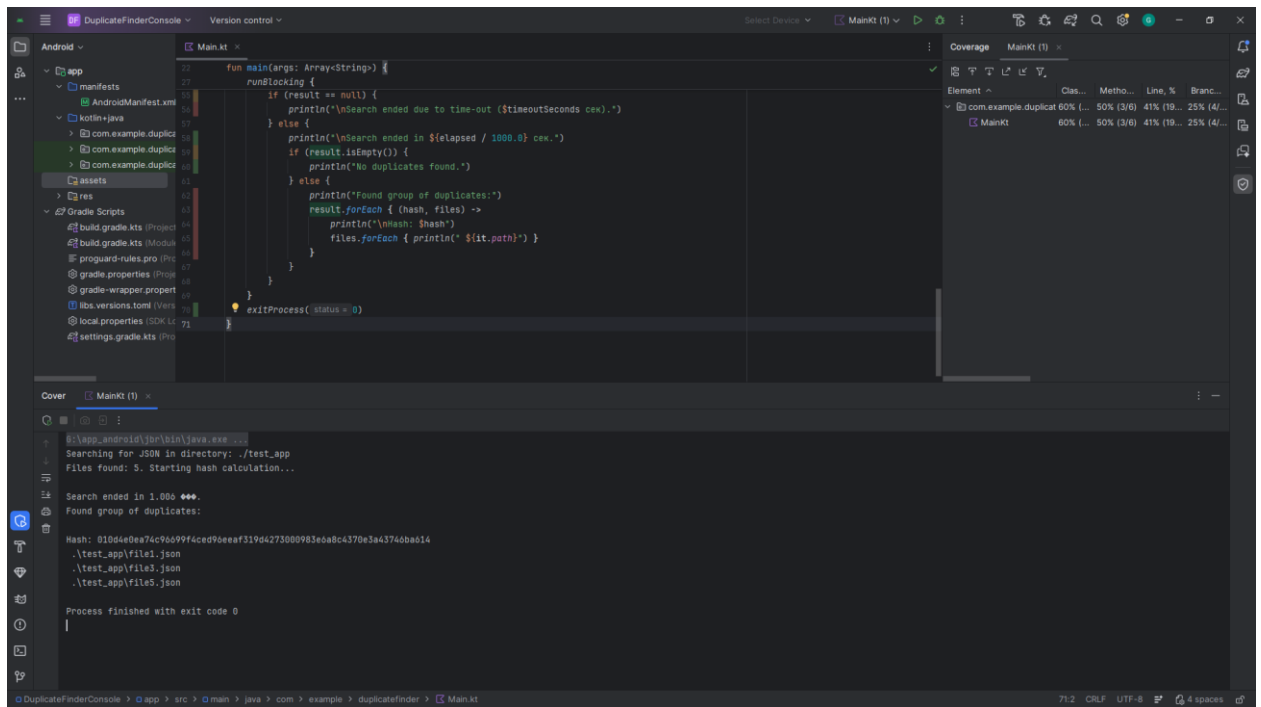


Рисунок 3 – Выполнение задания 2

```
package com.example.duplicatefinder
```

```
import kotlinx.coroutines.*
import java.io.File
import java.security.MessageDigest
import kotlin.system.exitProcess
```

```
suspend fun File.sha256(): String = withContext(Dispatchers.IO) {
    val digest = MessageDigest.getInstance("SHA-256")
    inputStream().use { fis ->
        val buffer = ByteArray(8192)
        var bytesRead: Int
        while (fis.read(buffer).also { bytesRead = it } != -1) {
```

```

        digest.update(buffer, 0, bytesRead)
    }
}
digest.digest().joinToString("") { "%02x".format(it) }
}

fun main(args: Array<String>) {
    val rootPath = if (args.isNotEmpty()) args[0] else "./test_app"
    val timeoutSeconds = 10L
    println("Searching for JSON in directory: $rootPath")
    val startTime = System.currentTimeMillis()
    runBlocking {
        val result = withTimeoutOrNull(timeoutSeconds * 1000) {
            val files = File(rootPath)
                .walkTopDown()
                .filter { it.isFile && it.extension.equals("json", ignoreCase = true) }
                .toList()
            if (files.isEmpty()) {
                println("JSON files were not found.")
                return@withTimeoutOrNull emptyMap()
            }
            println("Files found: ${files.size}. Starting hash calculation...")
            val deferredResults = files.map { file ->
                async {
                    try {
                        file.sha256()
                    } catch (e: Exception) {
                        println("File reading error ${file.path}: ${e.message}")
                        file to null
                    }
                }
            }
        }
        val results = deferredResults.awaitAll()
        results
            .filter { it.second != null }
            .groupBy({ it.second!! }, { it.first })
            .filter { it.value.size > 1 }
    }
    val elapsed = System.currentTimeMillis() - startTime
    if (result == null) {
        println("\nSearch ended due to time-out ($timeoutSeconds сек).")
    } else {
        println("\nSearch ended in ${elapsed / 1000.0} сек.")
        if (result.isEmpty()) {
            println("No duplicates found.")
        } else {

```

```

        println("Found group of duplicates:")
        result.forEach { (hash, files) ->
            println("\nHash: $hash")
            files.forEach { println(" ${it.path}") }
        }
    }
}
}
exitProcess(0)
}

```

Листинг 5 – MainActivity.kt

Задание 3: Экран поиска репозитория в GitHub с debounce

Debounce — это когда мы говорим функции: «Не выполняйся сразу. Подожди, пока меня перестанут дёргать хотя бы N миллисекунд. Тогда выполнись один раз с самыми свежими данными». Дополнительно можно почитать тут:

<https://kotlin.coroutines.ru/2025/12/25/%D1%83%D0%BF%D1%80%D0%B0%D0%B2%D0%BB%D0%B5%D0%BD%D0%B8%D0%B5-%D0%BF%D0%BE%D1%82%D0%BE%D0%BA%D0%BE%D0%BC-%D0%B2-kotlin-flow/>

Самая простая реализация выглядит так:

```
import kotlinx.coroutines.*
```

```

fun <T> CoroutineScope.debounce(
    waitMs: Long = 500L, destinationFunction: (T) -> Unit): (T) -> Unit {

    var debounceJob: Job? = null

    return { param: T ->

        debounceJob?.cancel()

        debounceJob = launch {

            delay(waitMs)

            destinationFunction(param)

        }
    }
}

```

Необходимо создать Android приложение.

Требования к UI:

- есть поле ввода;
- при вводе текста происходит «поиск» репозитория;
- результаты отображаются в LazyColumn;
- поиск запускается не на каждый символ, а с задержкой (debounce);
- во время поиска показывается индикатор загрузки;
- предыдущий поиск автоматически отменяется, если пользователь продолжает печатать;

Требования к корутинам:

- Использовать launch/async/withContext/delay;
- Реализовать debounce (можно с помощью delay + cancel);
- Отмена предыдущей задачи при новом вводе текста (через Job.cancel())
- Всё должно работать внутри rememberCoroutineScope()

Источник данных - большой json-файл github_repos.json в assets или res/raw (можно взять ~50–200 записей репозитория с полями: id, full_name, description, stargazers_count, language)

Пример структуры JSON файла:

```
{  
  "id": 123456,  
  "full_name": "JetBrains/kotlin",  
  "description": "The Kotlin Programming Language",  
  "stargazers_count": 48000,  
  "language": "Kotlin"  
}
```

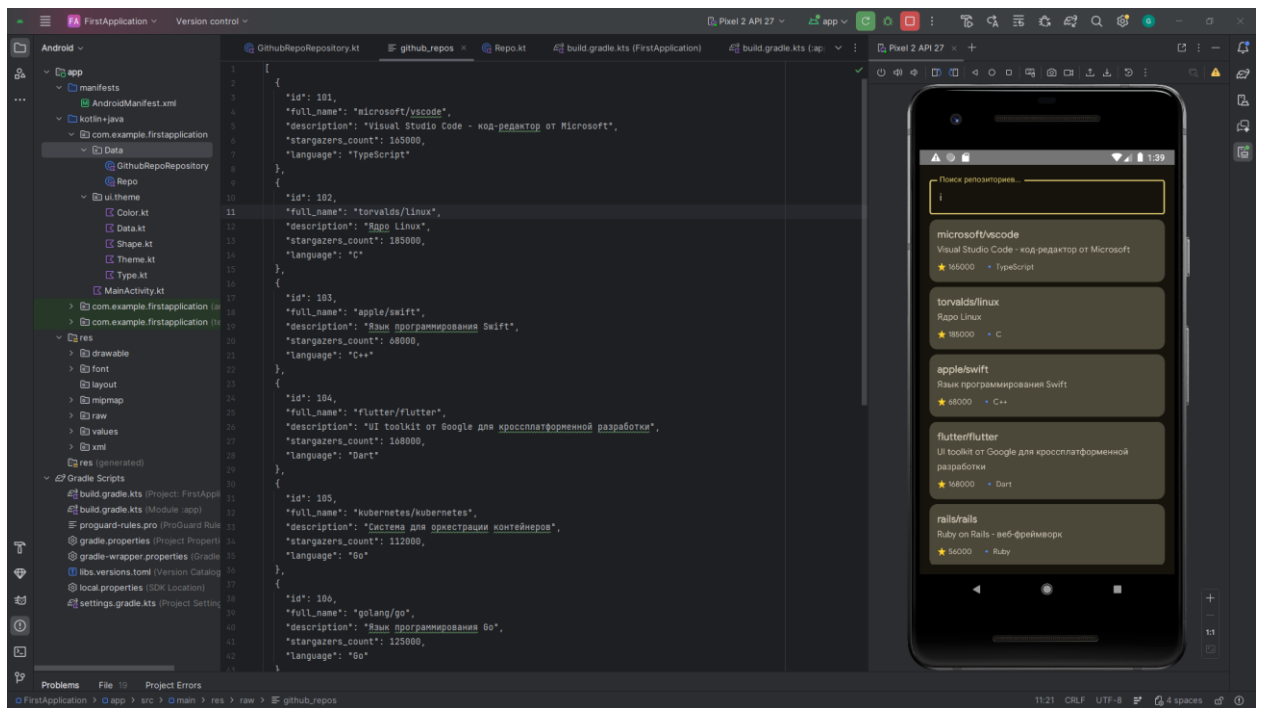


Рисунок 4 – Выполнение задания 3

Код на гите:

<https://github.com/oggiorgi/MIREA/tree/main/6sem/kotlin/4module/app/3code/FirstApplication>

Задание 4: Социальная лента с постами, аватарками и комментариями

Необходимо создать Android приложение. Требования к UI:

- при запуске экрана загружаются 8–15 постов из JSON;
- для каждого поста параллельно загружаются:
 - аватарка автора (имитация: delay + строка-цвет или url);
 - список комментариев (delay + фильтр по postId);
- посты отображаются по мере готовности (не ждём всех сразу);
- каждая карточка поста показывает своё состояние загрузки (Loading / Ready / Error);
- есть кнопка «Обновить» — при нажатии отменяются все текущие загрузки аватарок и комментариев;
- если комментарии или аватарка не загрузились - показывается placeholder / иконка ошибки;

Требования к корутинам:

- `rememberCoroutineScope()` для запуска задач из `composable`;
- `coroutineScope { ... }` или `supervisorScope { ... }` для параллельной загрузки подзадач одного поста;
- `async + await()` / `awaitAll()` для получения результатов;
- `Job.cancel()` для отмены всех дочерних задач при обновлении;
- `withContext(Dispatchers.IO)` для чтения JSON;
- искусственные `delay()` для имитации сети;
- обработка исключений (`try-catch` внутри `async`);

Источник данных - локальные json-файлы в `assets` или `res/raw`

1. `social_posts.json` — массив постов

```
[  
  {  
    "id": 1,  
    "userId": 1,  
    "title": "Красивый закат сегодня",  
    "body": "Смотрите, какой цвет неба...",  
    "avatarUrl": "https://i.pravatar.cc/150?img=1"  
  },  
  {  
    "id": 2,  
    "userId": 2,  
    "title": "Новый кофе-мастер-класс",  
    "body": "Приходите все!",  
    "avatarUrl": "https://i.pravatar.cc/150?img=2"
```

```

    }

    // ...

]

2. comments.json — все комментарии

[

{"postId": 1, "id": 101, "name": "Аня", "body": "Очень красиво! ✨"},

{"postId": 1, "id": 102, "name": "Макс", "body": "Где это снято?"},

{"postId": 2, "id": 201, "name": "Ольга", "body": "Записываюсь!"}

// ...

]
```

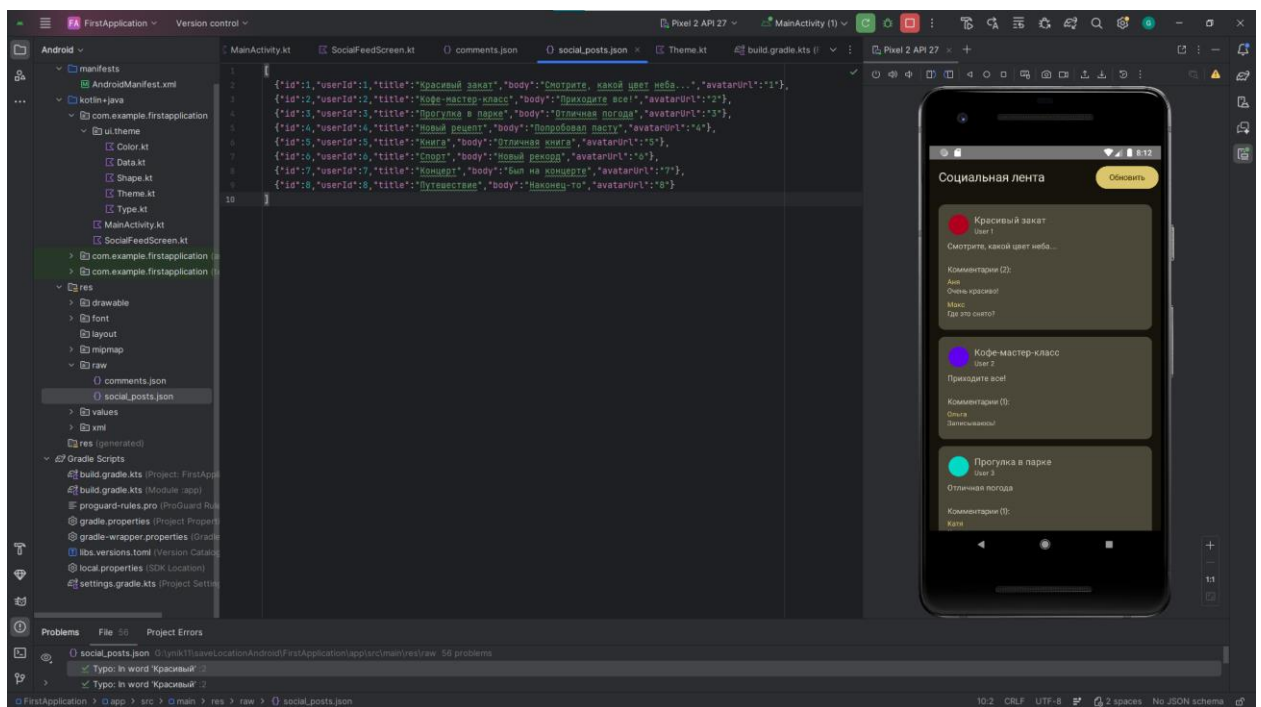


Рисунок 5 — Выполнение задания 4

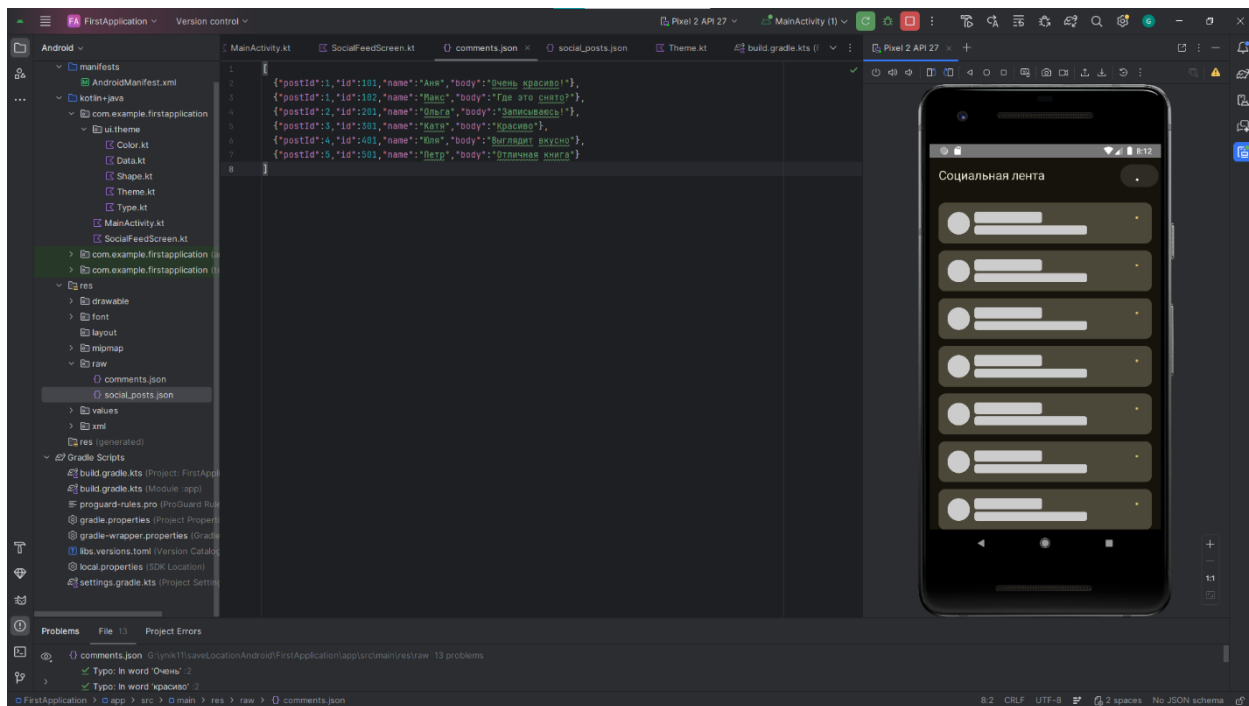


Рисунок 6 – Выполнение задания 4

```
package com.example.firstapplication

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Modifier
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.unit.dp
import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import androidx.lifecycle.viewmodel.compose.viewModel
import com.google.gson.Gson
import com.google.gson.reflect.TypeToken
import kotlinx.coroutines.*
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import java.io.InputStreamReader
import kotlin.random.Random
import androidx.compose.ui.graphics.asImageBitmap
import com.example.firstapplication.ui.theme.AppTheme
```



```

// ===== МОДЕЛИ =====
data class Post(val id: Int, val userId: Int, val title: String, val body: String, val
avatarUrl: String) // Модель поста
data class Comment(val postId: Int, val id: Int, val name: String, val body:
String) // Модель комментария

// ===== СОСТОЯНИЯ =====
sealed class PostState { // Состояния загрузки поста
    object Loading : PostState() // Загружается
    data class Ready(val post: Post, val avatar:
androidx.compose.ui.graphics.ImageBitmap?, val comments: List<Comment>) :
PostState() // Готов
    data class Error(val post: Post, val message: String) : PostState() // Ошибка
}

// ===== РЕПОЗИТОРИЙ =====
class Repository(private val context: android.content.Context) { // Работа с
данными
    private val gson = Gson() // Парсер JSON

    suspend fun loadPosts(): List<Post> = withContext(Dispatchers.IO) { //
Загрузка постов в фоне
        delay(500) // Имитация сети
        try {
            val reader =
InputStreamReader(context.resources.openRawResource(R.raw.social_posts)) //
Читаем файл
            val type = object : TypeToken<List<Post>>() {}.type // Тип для
парсинга
            gson.fromJson<List<Post>>(reader, type) // Парсим JSON
        } catch (e: Exception) { emptyList() } // При ошибке пустой список
    }

    suspend fun loadComments(): List<Comment> =
withContext(Dispatchers.IO) { // Загрузка комментариев
        delay(800) // Имитация сети
        try {
            val reader =
InputStreamReader(context.resources.openRawResource(R.raw.comments)) //
Читаем файл
            val type = object : TypeToken<List<Comment>>() {}.type
            gson.fromJson<List<Comment>>(reader, type) // Парсим
        } catch (e: Exception) { emptyList() }
    }
}

```

```

suspend fun loadAvatar(url: String):
androidx.compose.ui.graphics.ImageBitmap? = withContext(Dispatchers.IO) { //
Имитация аватарки
    delay(600 + Random.nextInt(300).toLong()) // Случайная задержка
    try {
        val colors = listOf(0xFF6200EE.toInt(), 0xFF03DAC5.toInt(),
0xFF3700B3.toInt(), 0xFF018786.toInt(), 0xFFB00020.toInt()) // Цвета
        val color = colors[Math.abs(url.hashCode()) % colors.size] // Выбор
цвета по URL

        val bitmap = android.graphics.Bitmap.createBitmap(100, 100,
android.graphics.Bitmap.Config.ARGB_8888) // Создаем картинку
        val canvas = android.graphics.Canvas(bitmap) // Холст для рисования
        val paint = android.graphics.Paint().apply { // Краска
            this.color = color
            isAntiAlias = true
        }
        canvas.drawCircle(50f, 50f, 45f, paint) // Рисуем круг

        bitmap.asImageBitmap() // Конвертируем в Compose Bitmap
    } catch (e: Exception) { null }
}

// ===== VIEWMODEL =====
class SocialViewModel(private val repository: Repository) : ViewModel() { //
Управление данными
    private val _posts = MutableStateFlow<Map<Int, PostState>>(emptyMap()) //
Состояние постов (внутреннее)
    val posts: StateFlow<Map<Int, PostState>> = _posts.asStateFlow() //
Состояние постов (внешнее)

    private val _refreshing = MutableStateFlow(false) // Флаг обновления
(внутренний)
    val refreshing: StateFlow<Boolean> = _refreshing.asStateFlow() // Флаг
обновления (внешний)

    private var loadJob: Job? = null // Текущая задача загрузки
    private var allComments = emptyList<Comment>() // Кэш комментариев

    fun loadPosts() { // Загрузка всех постов
        loadJob?.cancel() // Отмена предыдущей загрузки
        loadJob = viewModelScope.launch { // Запуск в скоупе ViewModel
            _refreshing.value = true // Включаем индикатор

```

```

_posts.value = emptyMap() // Очищаем старые посты

val posts = repository.loadPosts() // Загружаем посты
allComments = repository.loadComments() // Загружаем комментарии

_posts.value = posts.associate { it.id to PostState.Loading } // Ставим
статус Loading для всех

coroutineScope { // Область для параллельных задач
    posts.forEach { post -> // Для каждого поста
        launch { // Запускаем корутину
            supervisorScope { // Ошибка в одной не отменяет другие
                val avatarDeferred = async { // Асинхронно грузим аватар
                    try { repository.loadAvatar(post.avatarUrl) } catch (e:
Exception) { null }
                }

                val commentsDeferred = async { // Асинхронно грузим
комментарии
                    try {
                        delay(400) // Имитация сети
                        allComments.filter { it.postId == post.id } // Фильтр по
postId
                    } catch (e: Exception) { emptyList() }
                }

                try {
                    val avatar = avatarDeferred.await() // Ждем аватар
                    val comments = commentsDeferred.await() // Ждем
комментарии

                    _posts.value = _posts.value.toMutableMap().apply { //
Обновляем состояние
                        this[post.id] = if (avatar != null || comments.isNotEmpty())
// Если хоть что-то загружено
                            PostState.Ready(post, avatar, comments)
                            else PostState.Error(post, "Ошибка загрузки")
                        }
                    } catch (e: Exception) { // Ошибка загрузки
                        _posts.value = _posts.value.toMutableMap().apply {
                            this[post.id] = PostState.Error(post, e.message ?:
"Ошибка")
                        }
                    }
                }
            }
        }
    }
}

```



```
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.foundation.shape.CircleShape
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Person
import androidx.compose.material.icons.filled.Warning
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.unit.dp
import kotlinx.coroutines.launch
```

```
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun SocialFeedScreen(viewModel: SocialViewModel) {
    val posts by viewModel.posts.collectAsState()
    val refreshing by viewModel.refreshing.collectAsState()
    val scope = rememberCoroutineScope()
```

```
    LaunchedEffect(Unit) {
        viewModel.loadPosts()
    }
```

```
    MaterialTheme {
        Scaffold(
            topBar = {
                TopAppBar(
                    title = { Text("Социальная лента") },
                    actions = {
                        Button(
                            onClick = {
                                scope.launch {
                                    viewModel.refresh()
                                }
                            },
                            enabled = !refreshing
                        ) {
                            if (refreshing) {
                                CircularProgressIndicator(
                                    modifier = Modifier.size(20.dp),
                                    color = Color.White
                                )
                            }
                        }
                    }
                )
            }
        )
```

```

        )
        } else {
            Text("Обновить")
        }
    }
}
)
}
) { padding ->
    if (posts.isEmpty() && !refreshing) {
        Box(
            modifier = Modifier
                .fillMaxSize()
                .padding(padding),
            contentAlignment = Alignment.Center
        ) {
            Text("Нет постов")
        }
    } else {
        LazyColumn(
            modifier = Modifier
                .fillMaxSize()
                .padding(padding),
            contentPadding = PaddingValues(16.dp),
            verticalArrangement = Arrangement.spacedBy(12.dp)
        ) {
            items(posts.values.toList()) { state ->
                PostCard(state = state)
            }
        }
    }
}
}
}
}

@Composable
fun PostCard(state: PostState) {
    Card(
        modifier = Modifier.fillMaxWidth()
    ) {
        Column(
            modifier = Modifier.padding(16.dp)
        ) {
            when (state) {
                is PostState.Loading -> {

```

```

        Row(verticalAlignment = Alignment.CenterVertically) {
            Box(
                modifier = Modifier
                    .size(40.dp)
                    .clip(CircleShape)
                    .background(Color.LightGray)
            )
            Spacer(modifier = Modifier.width(8.dp))
            Column {
                Box(
                    modifier = Modifier
                        .width(120.dp)
                        .height(20.dp)
                        .background(Color.LightGray,
RoundedCornerShape(4.dp))
                )
                Spacer(modifier = Modifier.height(4.dp))
                Box(
                    modifier = Modifier
                        .width(200.dp)
                        .height(16.dp)
                        .background(Color.LightGray,
RoundedCornerShape(4.dp))
                )
            }
            Spacer(modifier = Modifier.weight(1f))
            CircularProgressIndicator(modifier = Modifier.size(24.dp))
        }
    }

    is PostState.Ready -> {
        // Аватар и заголовок
        Row(verticalAlignment = Alignment.CenterVertically) {
            if (state.avatar != null) {
                androidx.compose.foundation.Image(
                    bitmap = state.avatar,
                    contentDescription = null,
                    modifier = Modifier
                        .size(40.dp)
                        .clip(CircleShape)
                )
            } else {
                Box(
                    modifier = Modifier
                        .size(40.dp)

```

```

        .clip(CircleShape)
        .background(Color.Red),
        contentAlignment = Alignment.Center
    ) {
        Icon(
            imageVector = Icons.Default.Person,
            contentDescription = null,
            modifier = Modifier.size(24.dp),
            tint = Color.White
        )
    }
}

Spacer(modifier = Modifier.width(8.dp))

Column {
    Text(
        text = state.post.title,
        style = MaterialTheme.typography.titleMedium
    )
    Text(
        text = "User ${state.post.userId}",
        style = MaterialTheme.typography.bodySmall
    )
}

Spacer(modifier = Modifier.height(8.dp))
Text(
    text = state.post.body,
    style = MaterialTheme.typography.bodyMedium
)

// Комментарии
if (state.comments.isNotEmpty()) {
    Spacer(modifier = Modifier.height(12.dp))
    HorizontalDivider()
    Spacer(modifier = Modifier.height(8.dp))

    Text(
        text = "Комментарии (${state.comments.size}):",
        style = MaterialTheme.typography.titleSmall
    )

    state.comments.take(3).forEach { comment ->

```

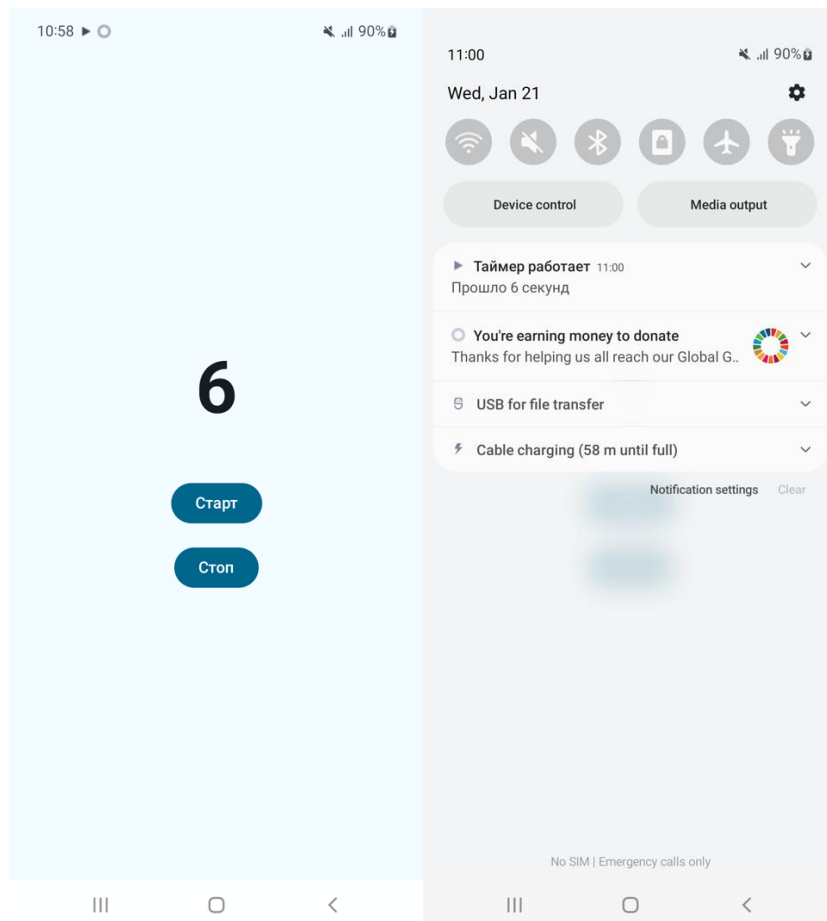

Часть II – Практическое задание

Задание 5 (Foreground service): Счётчик времени с уведомлением

Функционал приложения:

- При нажатии «Старт» запускается foreground-сервис;
- Сервис каждую секунду увеличивает счётчик (секунды);
- В уведомлении отображается: «Прошло X секунд»;
- Приложение показывает текущее значение крупным шрифтом;

Кнопка «Стоп» останавливает сервис.



Моя реализация:

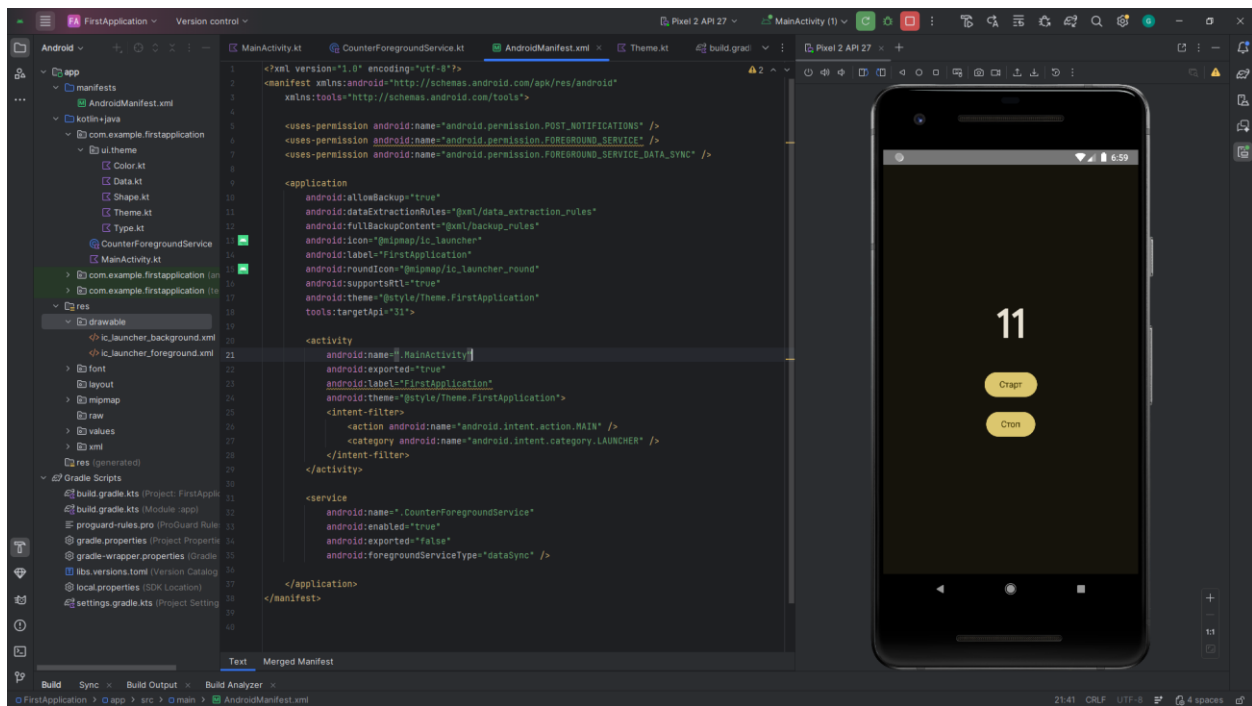


Рисунок 6 – Сервис в уведомлении

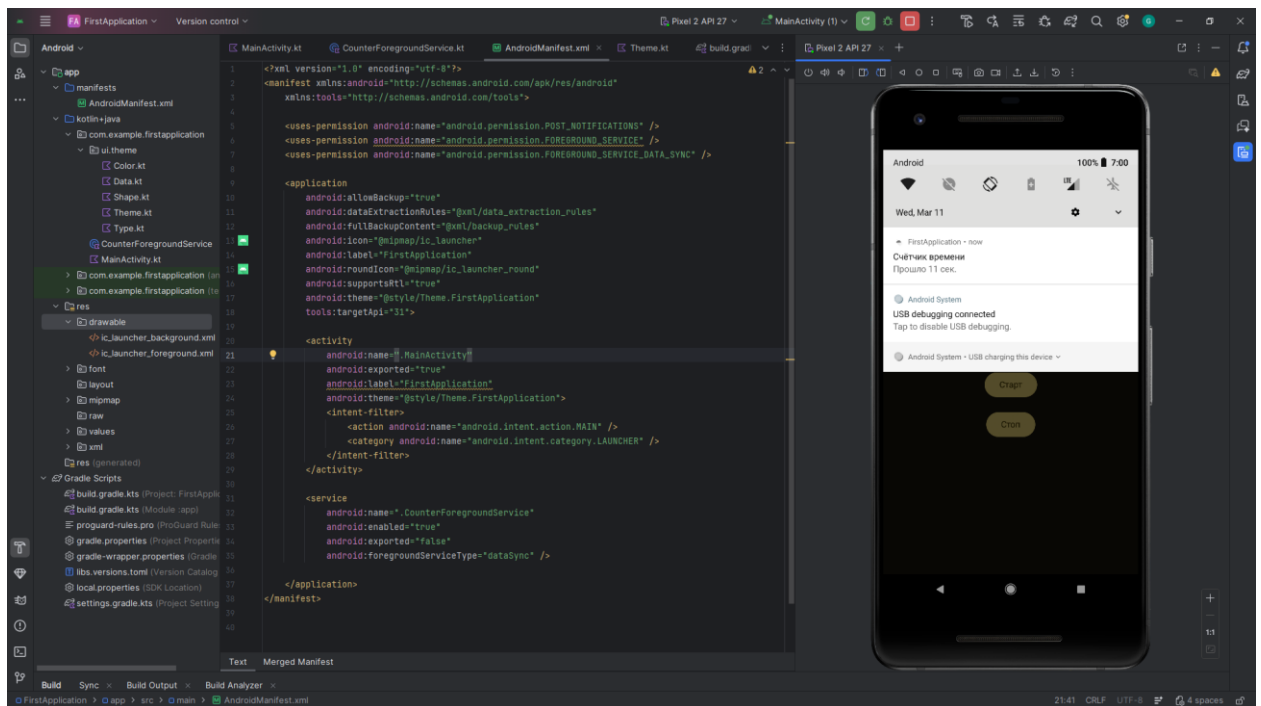


Рисунок 7 – UI таймер

```
package com.example.firstapplication

import android.Manifest
import android.content.BroadcastReceiver
import android.content.Context
import android.content.Intent
import android.content.IntentFilter
import android.content.pm.PackageManager
```

```
import android.os.Build
import android.os.Bundle
import android.util.Log
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.result.contract.ActivityResultContracts
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.height
import androidx.compose.material3.Button
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.core.content.ContextCompat
import com.example.firstapplication.ui.theme.AppTheme

class MainActivity : ComponentActivity() {
    private val tag = "MainActivity"

    private val requestPermissionLauncher = registerForActivityResult(
        ActivityResultContracts.RequestPermission()
    ) { isGranted ->
        if (isGranted) {
            Log.d(tag, "Notification permission granted")
        } else {
            Log.d(tag, "Notification permission denied")
        }
    }

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        // Запрос разрешения для уведомлений на Android 13+
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
            if (ContextCompat.checkSelfPermission(this,
Manifest.permission.POST_NOTIFICATIONS)
```

```

        != PackageManager.PERMISSION_GRANTED) {

requestPermissionLauncher.launch(Manifest.permission.POST_NOTIFICATIONS)
    }
}

setContent {
    AppTheme(darkTheme = true) {
        Surface(modifier = Modifier.fillMaxSize()) {
            TimerScreen()
        }
    }
}
}

@Composable
fun TimerScreen() {
    val context = LocalContext.current
    val counterState = remember { mutableIntStateOf(0) }

    DisposableEffect(context) {
        val receiver = object : BroadcastReceiver() {
            override fun onReceive(context: Context?, intent: Intent?) {
                if (intent?.action ==
CounterForegroundColorService.ACTION_COUNTER_UPDATE) {
                    val newValue =
intent.getIntExtra(CounterForegroundColorService.EXTRA_COUNTER, 0)
                    counterState.intValue = newValue
                }
            }
        }

        val filter =
IntentFilter(CounterForegroundColorService.ACTION_COUNTER_UPDATE)
        ContextCompat.registerReceiver(
            context,
            receiver,
            filter,
            ContextCompat.RECEIVER_NOT_EXPORTED
        )

        onDispose {
            context.unregisterReceiver(receiver)

```

```

    }
}

CounterApp(
    counter = counterState.intValue,
    onStartClick = {
        val intent = Intent(context, CounterForegroundService::class.java)
        ContextCompat.startForegroundService(context, intent)
    },
    onStopClick = {
        val intent = Intent(context, CounterForegroundService::class.java)
        context.stopService(intent)
    }
)
}

@Composable
fun CounterApp(counter: Int, onStartClick: () -> Unit, onStopClick: () -> Unit) {
    Column(
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center,
        modifier = Modifier.fillMaxSize()
    ) {
        Text(
            text = counter.toString(),
            fontSize = 64.sp,
            fontWeight = FontWeight.Bold
        )

        Spacer(modifier = Modifier.height(32.dp))

        Button(onClick = onStartClick) {
            Text("Старт")
        }

        Spacer(modifier = Modifier.height(16.dp))

        Button(onClick = onStopClick) {
            Text("Стоп")
        }
    }
}
}

```

Листинг 8 - MainActivity.kt

```
package com.example.firstapplication

import android.app.NotificationChannel
import android.app.NotificationManager
import android.app.Service
import android.content.Intent
import android.content.pm.ServiceInfo
import android.os.Build
import android.os.Handler
import android.os.IBinder
import android.os.Looper
import androidx.core.app.NotificationCompat
import androidx.core.app.ServiceCompat

class CounterForegroundService : Service() {
    private val channelId = "counter_channel"
    private val notificationId = 1
    private var counter = 0
    private val handler = Handler(Looper.getMainLooper())

    private val updateRunnable = object : Runnable {
        override fun run() {
            counter++
            updateNotification()
            sendBroadcastUpdate()
            handler.postDelayed(this, 1000)
        }
    }

    companion object {
        const val ACTION_COUNTER_UPDATE =
            "com.example.firstapplication.UPDATE"
        const val EXTRA_COUNTER = "counter"
    }

    override fun onCreate() {
        super.onCreate()
        createNotificationChannel()
    }

    override fun onStartCommand(intent: Intent?, flags: Int, startId: Int): Int {
        startForegroundServiceWithNotification()
        handler.post(updateRunnable)
        return START_NOT_STICKY
    }
}
```

```

override fun onBind(intent: Intent?): IBinder? = null

override fun onDestroy() {
    handler.removeCallbacks(updateRunnable)
    stopForeground(true)
    super.onDestroy()
}

private fun createNotificationChannel() {
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
        val channel = NotificationChannel(
            channelId,
            "Счётчик времени",
            NotificationManager.IMPORTANCE_LOW
        ).apply {
            description = "Показывает текущее значение счётчика"
        }
        val manager = getSystemService(NOTIFICATION_SERVICE) as
NotificationManager
        manager.createNotificationChannel(channel)
    }
}

private fun startForegroundServiceWithNotification() {
    val notification = NotificationCompat.Builder(this, channelId)
        .setContentTitle("Счётчик времени")
        .setContentText("Прошло $counter сек.")
        .setSmallIcon(android.R.drawable.ic_dialog_info) // Используем
системную иконку
        .setPriority(NotificationCompat.PRIORITY_LOW)
        .build()

    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.Q) {
        ServiceCompat.startForeground(
            this,
            notificationId,
            notification,
            ServiceInfo.FOREGROUND_SERVICE_TYPE_DATA_SYNC
        )
    } else {
        startForeground(notificationId, notification)
    }
}

```



```

private fun updateNotification() {
    val notification = NotificationCompat.Builder(this, channelId)
        .setContentTitle("Счётчик времени")
        .setContentText("Прошло $counter сек.")
        .setSmallIcon(R.drawable.ic_launcher_foreground)
        .build()

    val manager = getSystemService(NOTIFICATION_SERVICE) as
NotificationManager
    manager.notify(notificationId, notification)
}

private fun sendBroadcastUpdate() {
    val intent = Intent(ACTION_COUNTER_UPDATE).apply {
        putExtra(EXTRA_COUNTER, counter)
    }
    sendBroadcast(intent)
}
}

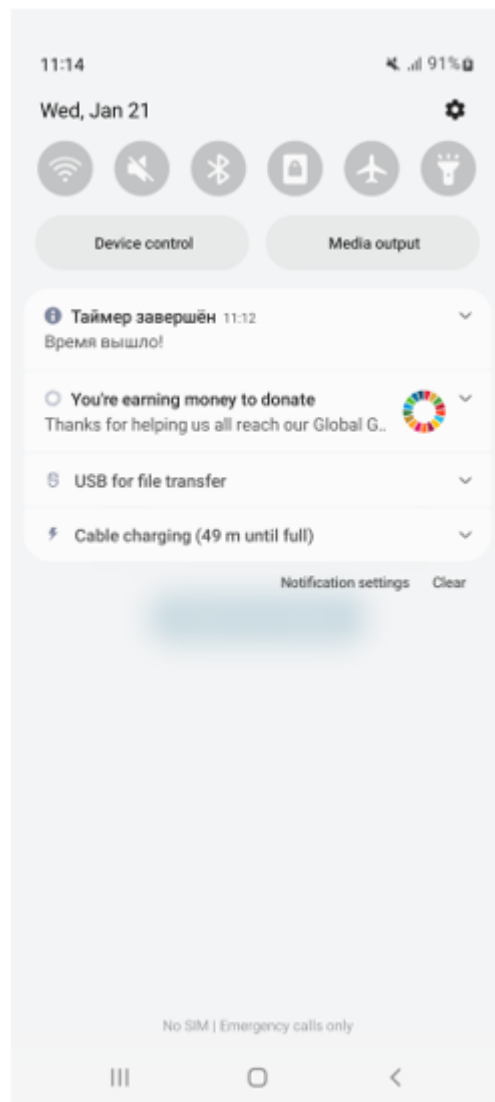
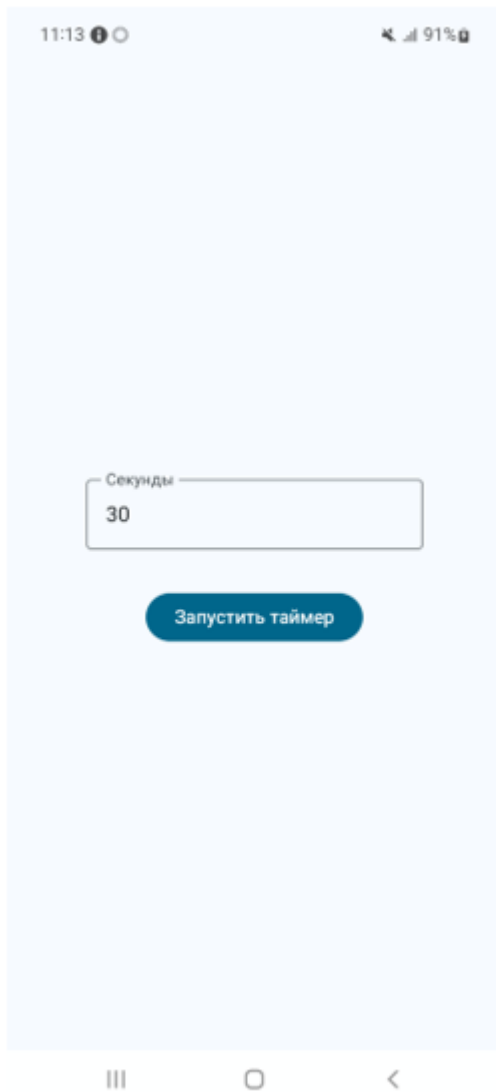
```

Листинг 9 - CounterForegroundService.kt

Задание 6 (Background service): Одноразовый таймер с уведомлением

Функционал приложения:

- Пользователь вводит количество секунд (например: 30);
- Нажимает «Запустить таймер»;
- Сервис ждёт указанное время и показывает уведомление «Таймер завершён!» с сообщением «Время вышло»;
- Сервис сам себя останавливает.



Моя реализация:

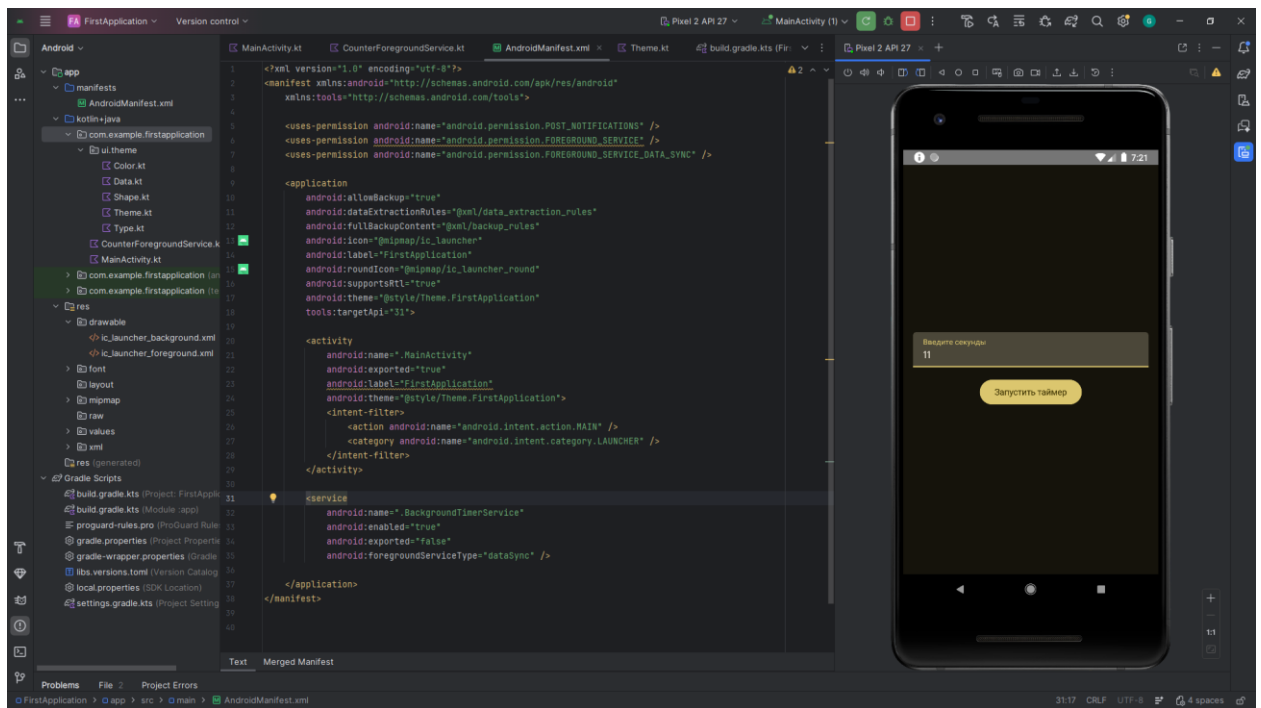


Рисунок 8 – Интерфейс приложения

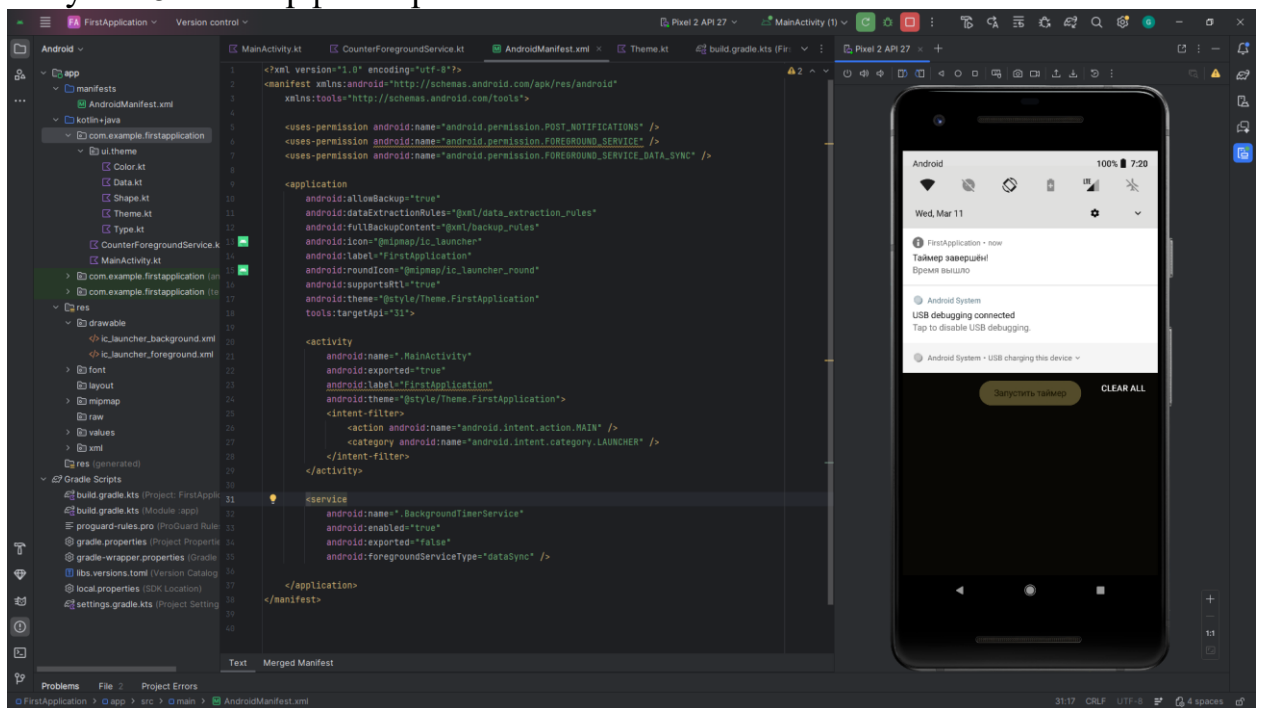


Рисунок 9 – Уведомление по окончанию таймера

```
package com.example.firstapplication

import android.content.Intent
import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
```

```

import androidx.compose.foundation.layout.*
import androidx.compose.foundation.text.KeyboardOptions
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.text.input.KeyboardType
import androidx.compose.ui.unit.dp
import com.example.firstapplication.ui.theme.AppTheme

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            AppTheme(darkTheme = true) {
                Surface(
                    modifier = Modifier.fillMaxSize()
                ){
                    TimerScreen()
                }
            }
        }
    }
}

@Composable
fun TimerAppTheme(content: @Composable () -> Unit) {
    AppTheme (content = content)
}

@Composable
fun TimerScreen() {
    var secondsInput by remember { mutableStateOf("") }
    val context = LocalContext.current
    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(16.dp),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center
    ) {
        TextField(
            value = secondsInput,
            onValueChange = { newValue ->
                secondsInput = newValue.filter { it.isDigit() }
            },
        ),
    }
}

```

```

        label = { Text("Введите секунды") },
        keyboardOptions = KeyboardOptions(keyboardType =
KeyboardType.Number),
        singleLine = true,
        modifier = Modifier.fillMaxWidth()
    )
    Spacer(modifier = Modifier.height(16.dp))
    Button(
        onClick = {
            val seconds = secondsInput.toIntOrNull()
            if (seconds != null && seconds > 0) {
                val intent = Intent(context,
BackgroundTimerService::class.java).apply {
                    putExtra("seconds", seconds)
                }
                context.startService(intent)
            }
        },
        enabled = (secondsInput.toIntOrNull() ?: 0) > 0
    ) {
        Text("Запустить таймер")
    }
}
}

```

Листинг 9 - MainActivity.kt

```

package com.example.firstapplication

import android.app.NotificationChannel
import android.app.NotificationManager
import android.app.Service
import android.content.Intent
import android.os.IBinder
import androidx.core.app.NotificationCompat
import kotlinx.coroutines.*

class BackgroundTimerService : Service() {
    private val serviceScope = CoroutineScope(Dispatchers.IO + SupervisorJob())
    private lateinit var notificationManager: NotificationManager
    private val channelId = "timer_channel"
    private val notificationId = 1
    override fun onCreate() {
        super.onCreate()
        notificationManager = getSystemService(NOTIFICATION_SERVICE) as

```

```

NotificationManager
    createNotificationChannel()
}

override fun onStartCommand(intent: Intent?, flags: Int, startId: Int): Int {
    val seconds = intent?.getIntExtra("seconds", 0) ?: 0
    if (seconds > 0) {
        serviceScope.launch {
            delay(seconds * 1000L)
            showNotification()
            stopSelf()
        }
    } else {
        stopSelf()
    }
    return START_NOT_STICKY
}

private fun showNotification() {
    val notification = NotificationCompat.Builder(this, channelId)
        .setContentTitle("Таймер завершён!")
        .setContentText("Время вышло")
        .setSmallIcon(android.R.drawable.ic_dialog_info)
        .setPriority(NotificationCompat.PRIORITY_DEFAULT)
        .build()
    notificationManager.notify(notificationId, notification)
}

private fun createNotificationChannel() {
    val channel = NotificationChannel(
        channelId,
        "Timer Channel",
        NotificationManager.IMPORTANCE_DEFAULT
    )
    notificationManager.createNotificationChannel(channel)
}

override fun onBind(intent: Intent?): IBinder? = null
override fun onDestroy() {
    serviceScope.cancel()
    super.onDestroy()
}
}

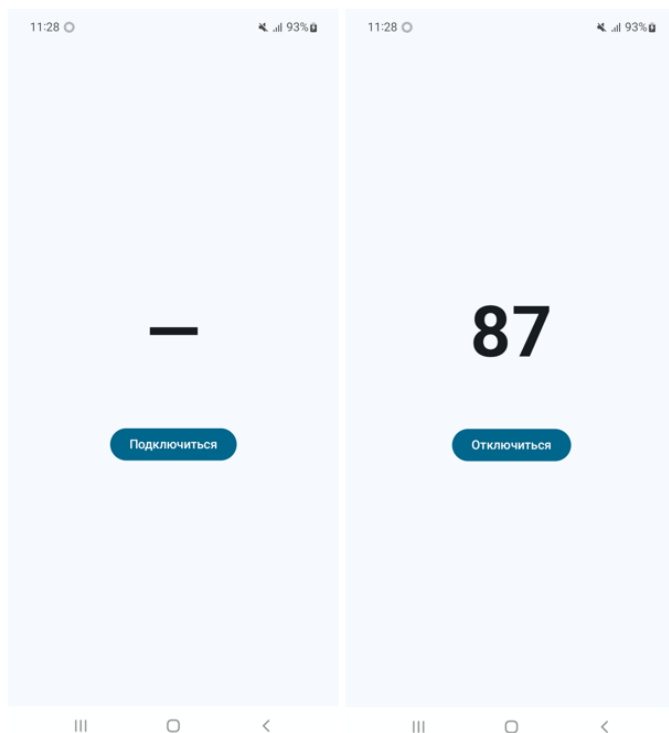
```

Листинг 10 - CounterForegroundService.kt

Задание 7 (Bind service): Случайное число каждую секунду

Функционал приложения:

- Приложение подключается к сервису;
- Сервис каждую секунду генерирует случайное число 0 ..100;
- UI показывает последнее число;
- Кнопки «Подключиться» / «Отключиться».



Моя реализация:

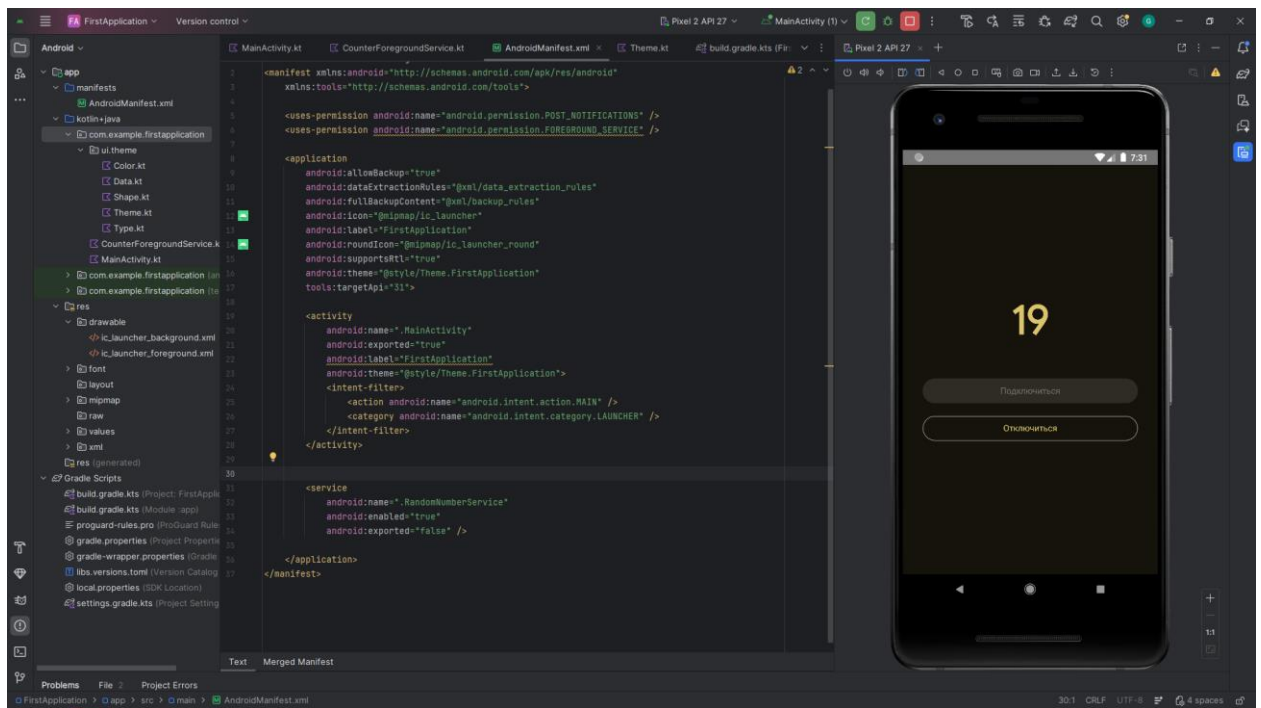


Рисунок 10 – Когда подключен

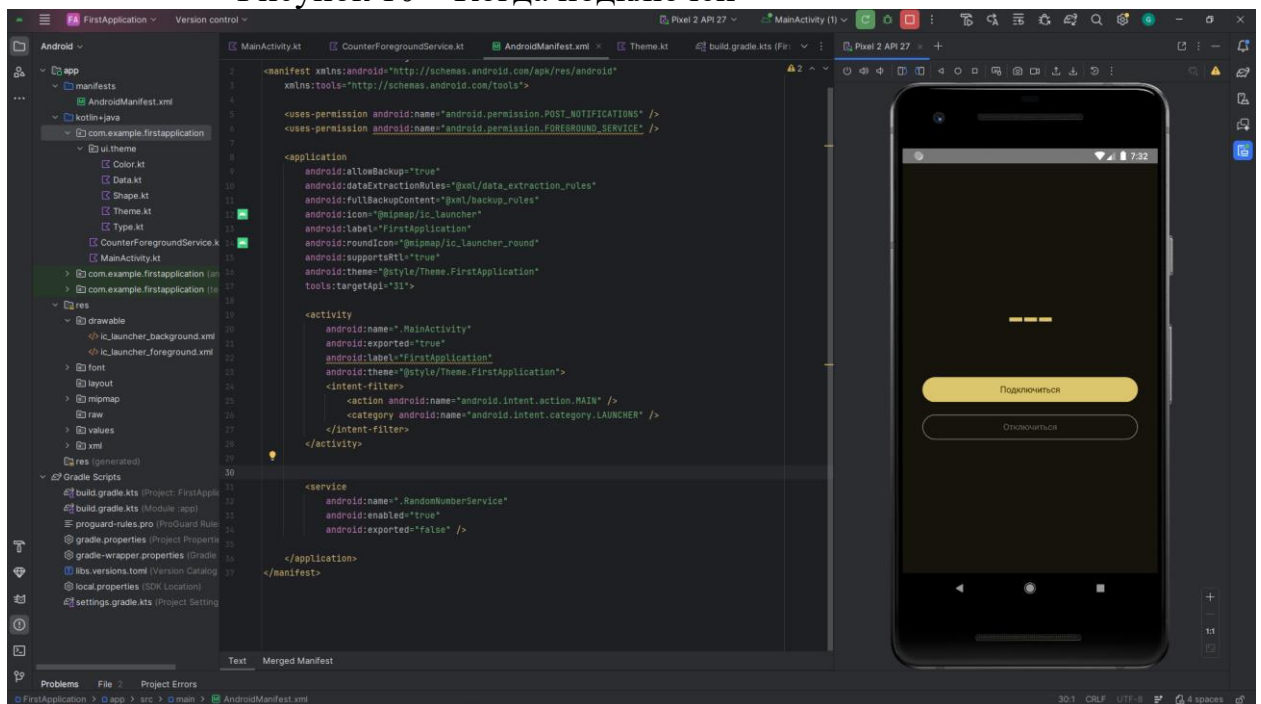


Рисунок 11 – Когда отключен

```

package com.example.firstapplication

import android.app.Service
import android.content.Intent
import android.os.Binder

```



```
import android.os.IBinder
import kotlinx.coroutines.*
class RandomNumberService : Service() {
    private val binder = RandomBinder()
    private var listener: ((Int) -> Unit)? = null

    private val serviceScope = CoroutineScope(Dispatchers.Default +
SupervisorJob())
    private var generateJob: Job? = null

    inner class RandomBinder : Binder() {
        fun getService(): RandomNumberService = this@RandomNumberService
    }
    override fun onBind(intent: Intent?): IBinder {
        return binder
    }
    fun setListener(listener: ((Int) -> Unit)? ) {
        this.listener = listener
        if (listener != null) {
            startGenerating()
        } else {
            stopGenerating()
        }
    }
    private fun startGenerating() {
        if (generateJob != null) return
        generateJob = serviceScope.launch {
            while (isActive) {
                val nextNumber = (0..100).random()
                withContext(Dispatchers.Main) {
                    listener?.invoke(nextNumber)
                }
                delay(1000)
            }
        }
    }
    private fun stopGenerating() {
        generateJob?.cancel()
        generateJob = null
    }
    override fun onDestroy() {
        super.onDestroy()
        stopGenerating()
        serviceScope.cancel()
    }
}
```

```
}  
}
```

Листинг 11 - CounterForegroundService.kt

```
package com.example.firstapplication  
  
import android.content.ComponentName  
import android.content.Intent  
import android.content.ServiceConnection  
import android.os.Bundle  
import android.os.IBinder  
import androidx.activity.ComponentActivity  
import androidx.activity.compose.setContent  
import androidx.compose.foundation.layout.*  
import androidx.compose.material3.*  
import androidx.compose.runtime.*  
import androidx.compose.ui.Alignment  
import androidx.compose.ui.Modifier  
import androidx.compose.ui.text.font.FontWeight  
import androidx.compose.ui.unit.dp  
import androidx.compose.ui.unit.sp  
import com.example.firstapplication.ui.theme.AppTheme  
  
class MainActivity : ComponentActivity() {  
    private var randomService by  
mutableStateOf<RandomNumberService?>(null)  
    private var isBound by mutableStateOf(false)  
    private val connection = object : ServiceConnection {  
        override fun onServiceConnected(name: ComponentName?, service:  
IBinder?) {  
            val binder = service as RandomNumberService.RandomBinder  
            randomService = binder.getService()  
            isBound = true  
        }  
        override fun onServiceDisconnected(name: ComponentName?) {  
            randomService = null  
            isBound = false  
        }  
    }  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContent {  
            AppTheme(darkTheme = true) {  
                Surface(  

```

```

        modifier = Modifier.fillMaxSize(),
        color = MaterialTheme.colorScheme.background
    ) {
        RandomNumberScreen(
            onBind = { bindToService() },
            onUnbind = { unbindFromService() },
            isBound = isBound,
            service = randomService
        )
    }
}

private fun bindToService() {
    val intent = Intent(this, RandomNumberService::class.java)
    bindService(intent, connection, BIND_AUTO_CREATE)
}

private fun unbindFromService() {
    if (isBound) {
        randomService?.setListener(null)
        unbindService(connection)
        isBound = false
        randomService = null
    }
}

override fun onStop() {
    super.onStop()
    if (isBound) {
        unbindFromService()
    }
}

@Composable
fun RandomNumberScreen(
    onBind: () -> Unit,
    onUnbind: () -> Unit,
    isBound: Boolean,
    service: RandomNumberService?
) {
    var currentNumber by remember { mutableIntStateOf(0) }
    LaunchedEffect(service) {
        service?.setListener { number ->
            currentNumber = number
        }
    }
}

```

```

Column(
    modifier = Modifier
        .fillMaxSize()
        .padding(32.dp),
    horizontalAlignment = Alignment.CenterHorizontally,
    verticalArrangement = Arrangement.Center
) {
    Text(
        text = if (isBound) currentNumber.toString() else "---",
        fontSize = 64.sp,
        fontWeight = FontWeight.Bold,
        color = MaterialTheme.colorScheme.primary
    )
    Spacer(modifier = Modifier.height(48.dp))
    Button(
        onClick = onBind,
        enabled = !isBound,
        modifier = Modifier.fillMaxWidth()
    ) {
        Text("Подключиться")
    }
    Spacer(modifier = Modifier.height(12.dp))
    OutlinedButton(
        onClick = onUnbind,
        enabled = isBound,
        modifier = Modifier.fillMaxWidth()
    ) {
        Text("Отключиться")
    }
}
}

```

Листинг 12 - MainActivity.kt

Задание 8 (Последовательная цепочка): Обработка фото перед загрузкой в облако

Функционал приложения:

Пользователь выбрал фото - приложение должно выполнить следующий набор действий:

1. Сжать фото (Worker 1);
2. Добавить водяной знак или подпись (Worker 2);
3. Загрузить готовое фото в облако (Worker 3, имитация).

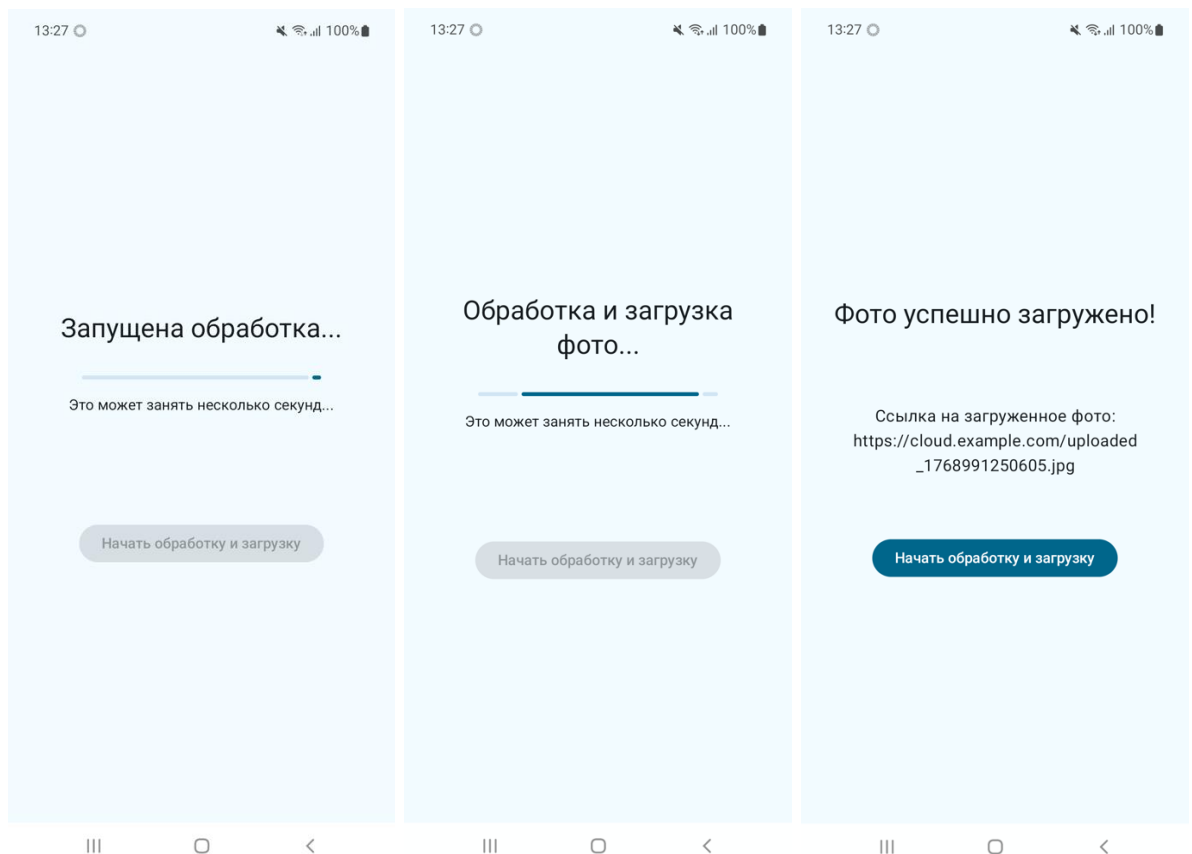
Каждый шаг зависит от предыдущего - последовательная цепочка.

Требования к реализации:

- Одна кнопка «Начать обработку и загрузку»;
- Показывать текущий шаг крупным текстом (например: «Сжимаем фото...», «Добавляем водяной знак...», «Загружаем...»);
- Показывать прогресс-бар (имитация 0–100% для каждого шага);
- В конце — сообщение «Готово! Фото загружено» + путь/имя файла;
- Если любой шаг упал - показать ошибку и отменить всю цепочку;

Минимальный UI:

- Text (headline) — текущий статус;
- LinearProgressIndicator (когда работает);
- Button «Начать обработку» (disabled во время работы);
- Text — результат или ошибка.



Моя реализация:

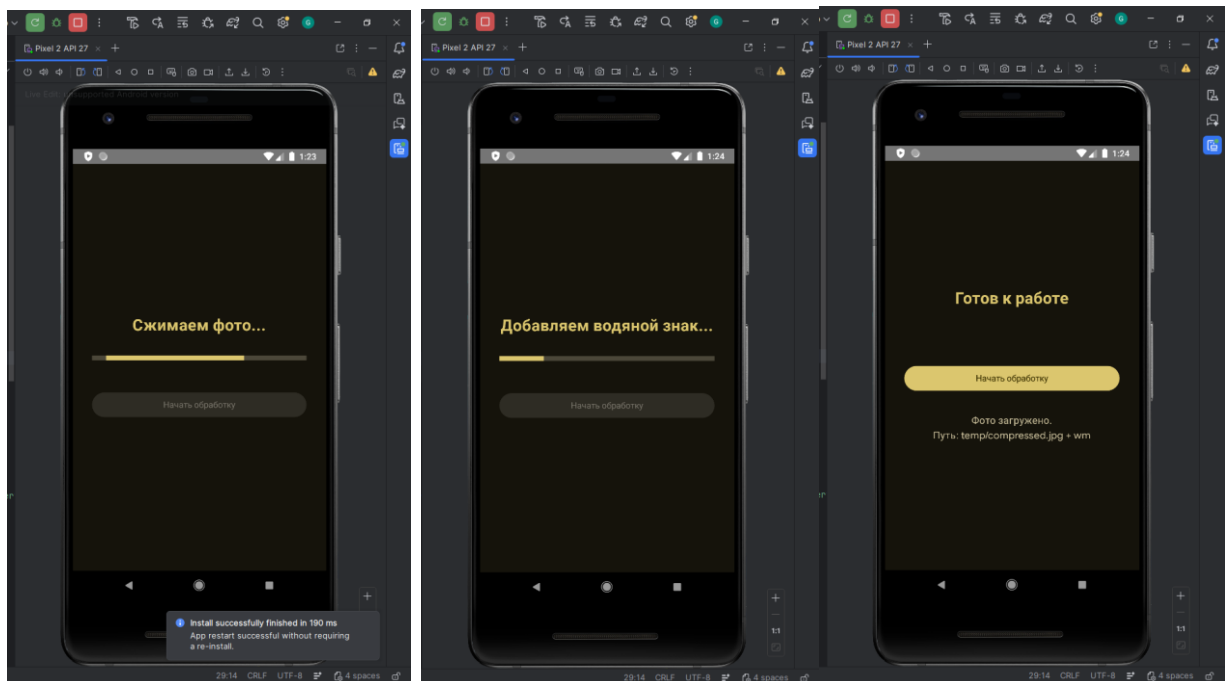


Рисунок 12 – Задание 8

```
package com.example.firstapplication
```

```

import android.content.Context
import androidx.work.CoroutineWorker
import androidx.work.WorkerParameters
import androidx.work.workDataOf
import kotlinx.coroutines.delay
class CompressWorker(context: Context, params: WorkerParameters) :
    CoroutineWorker(context, params) {
    override suspend fun doWork(): Result {
        try {
            setProgress(workDataOf("status" to "Сжимаем фото..."))
            delay(3000)
            return Result.success(workDataOf("image_path" to
"temp/compressed.jpg"))
        } catch (_: Exception) {
            return Result.failure()
        }
    }
}

```

Листинг 13 – CompressWorker.kt

```

package com.example.firstapplication

import android.content.Context
import androidx.work.CoroutineWorker
import androidx.work.WorkerParameters
import androidx.work.workDataOf
import kotlinx.coroutines.delay
class WatermarkWorker(context: Context, params: WorkerParameters) :
    CoroutineWorker(context, params) {
    override suspend fun doWork(): Result {
        try {
            setProgress(workDataOf("status" to "Добавляем водяной знак..."))
            delay(3000)
            val path = inputData.getString("image_path")
            return Result.success(workDataOf("image_path" to "$path + wm"))
        } catch (_: Exception) {
            return Result.failure()
        }
    }
}

```

Листинг 14 – WatermarkWorker.kt

```

package com.example.firstapplication

import android.content.Context
import androidx.work.CoroutineWorker

```

```

import androidx.work.WorkerParameters
import androidx.work.workDataOf
import kotlinx.coroutines.delay
class UploadWorker(context: Context, params: WorkerParameters) :
CoroutineWorker(context, params) {
    override suspend fun doWork(): Result {
        try {
            setProgress(workDataOf("status" to "Загружаем..."))
            delay(3000)
            val path = inputData.getString("image_path")
            return Result.success(workDataOf("final_result" to "Фото
загружено.\nПуть: $path"))
        } catch (_: Exception) {
            return Result.failure()
        }
    }
}

```

Листинг 15 – UploadWorker.kt

```

package com.example.firstapplication

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.*
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.runtime.livedata.observeAsState
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.style.TextAlign
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.work.*
import com.example.firstapplication.ui.theme.AppTheme

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            AppTheme(darkTheme = true) {
                Surface(
                    modifier = Modifier.fillMaxSize()
                ){
                    WorkManagerScreen()
                }
            }
        }
    }
}

```



```

    }
    }
    }
}

@Composable
fun WorkManagerScreen() {
    val context = androidx.compose.ui.platform.LocalContext.current
    val workManager = remember { WorkManager.getInstance(context) }
    val workInfos by
workManager.getWorkInfosForUniqueWorkLiveData("photo_processing").observeAsState()
    val activeWorkInfo = workInfos?.find { it.state == WorkInfo.State.RUNNING }
    val isRunning = workInfos?.any {
        it.state == WorkInfo.State.RUNNING || it.state ==
WorkInfo.State.ENQUEUED
    } == true
    val statusText = activeWorkInfo?.progress?.getString("status") ?:
"Ожидание..."
    val finalResult = workInfos?.find { it.outputData.getString("final_result") !=
null }
        ?.outputData?.getString("final_result")
    val hasError = workInfos?.any { it.state == WorkInfo.State.FAILED } == true
    Column(
        modifier = Modifier.fillMaxSize().padding(32.dp),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center
    ) {
        Text(
            text = if (hasError) "Ошибка!" else if (isRunning) statusText else "Готов
к работе",
            fontSize = 26.sp,
            fontWeight = FontWeight.Bold,
            textAlign = TextAlign.Center,
            color = if (hasError) MaterialTheme.colorScheme.error else
MaterialTheme.colorScheme.primary
        )
        Spacer(modifier = Modifier.height(32.dp))
        if (isRunning) {
            LinearProgressIndicator(
                modifier = Modifier.fillMaxWidth().height(8.dp)
            )
        } else {

```

```

        Spacer(modifier = Modifier.height(8.dp))
    }
    Spacer(modifier = Modifier.height(48.dp))
    Button(
        onClick = {
            val compress =
OneTimeWorkRequestBuilder<CompressWorker>().build()
            val watermark =
OneTimeWorkRequestBuilder<WatermarkWorker>().build()
            val upload = OneTimeWorkRequestBuilder<UploadWorker>().build()
            workManager.beginUniqueWork("photo_processing",
ExistingWorkPolicy.REPLACE, compress)
                .then(watermark)
                .then(upload)
                .enqueue()
        },
        enabled = !isRunning,
        modifier = Modifier.fillMaxWidth()
    ) {
        Text("Начать обработку")
    }
    if (finalResult != null && !isRunning && !hasError) {
        Spacer(modifier = Modifier.height(32.dp))
        Text(
            text = finalResult,
            fontSize = 16.sp,
            color = MaterialTheme.colorScheme.secondary,
            textAlign = TextAlign.Center
        )
    }
}
}
}

```

Листинг 16 – MainActivity.kt

Задание 9 (Параллельная обработка): Загрузка нескольких файлов и финальная сборка итогового отчета о погоде

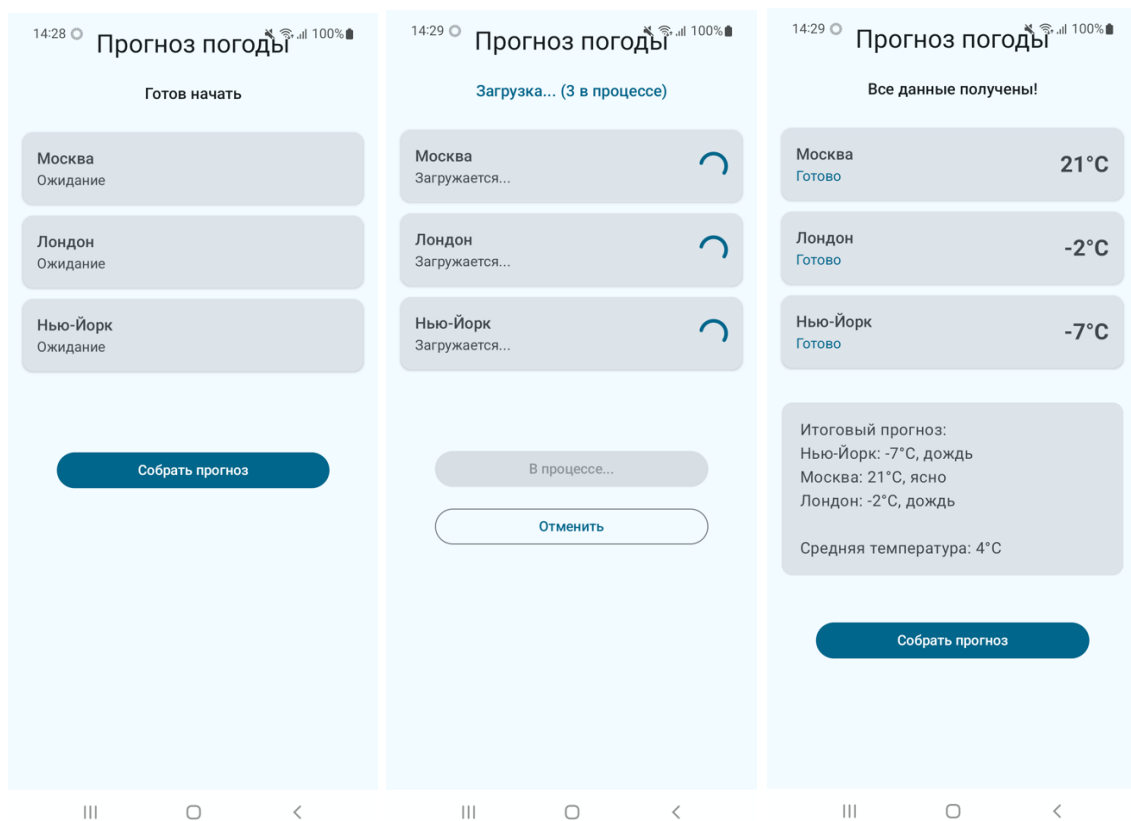
Функционал приложения:

Приложение собирает прогноз погоды для 3–4 городов одновременно (параллельно). Пользователь нажимает кнопку - начинается фоновая работа. Приложение показывает уведомление в статус-баре, которое обновляется по мере выполнения:

- «Загружаем погоду для 3 городов...»;
- «Готово: Москва и Лондон, Нью-Йорк в процессе...»;
- «Все данные получены, формируем отчёт...»;
- «Отчёт готов! Средняя температура +12°C».

Требования к реализации:

- Кнопка «Собрать прогноз»;
- После нажатия кнопки - сразу показывается **постоянное уведомление** (ongoing) с прогрессом;
- Уведомление обновляется каждые несколько секунд (или по мере завершения Worker'ов);
- Используется **Foreground Service** + WorkManager (чтобы уведомление могло жить долго);
- Или просто **WorkManager** с **setForegroundAsync** внутри Worker'ов (рекомендуемый способ с WorkManager 2.8+);
- Финальный Worker объединяет результаты и обновляет уведомление в последний раз;



Моя реализация:

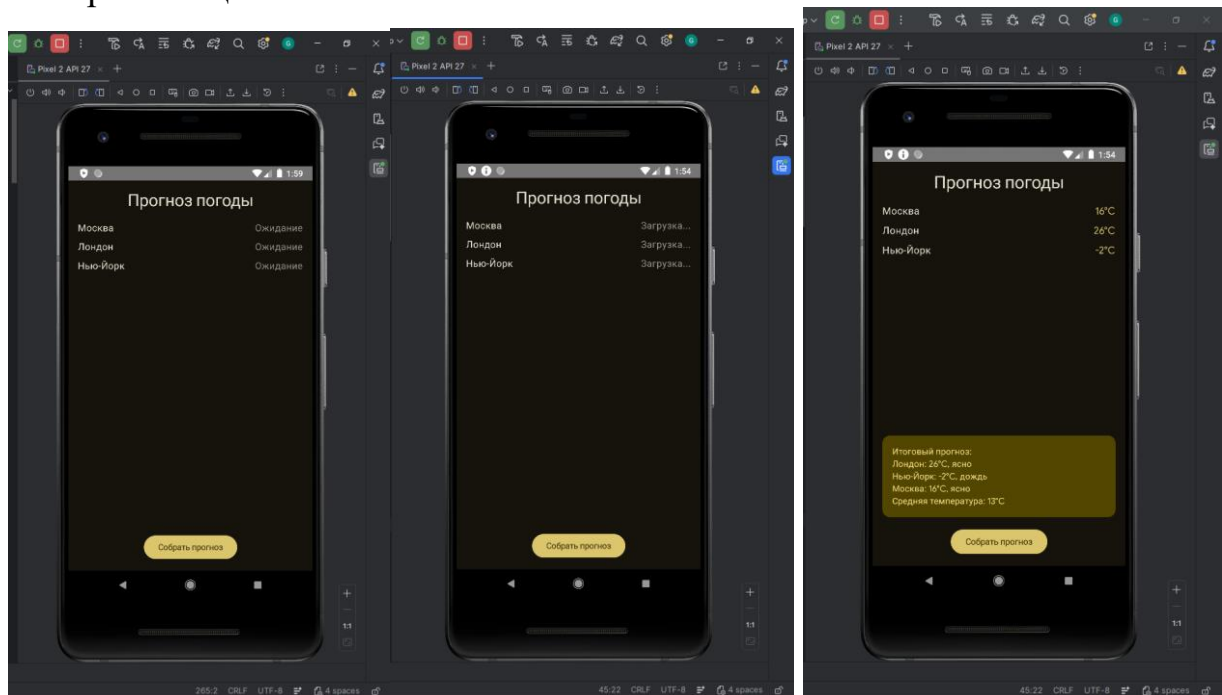


Рисунок 13 – Задание 9

Код на гите:

[https://github.com/oggiorgi/MIREA/tree/main/6sem/kotlin/4module/app/9code/Fir
stApplication](https://github.com/oggiorgi/MIREA/tree/main/6sem/kotlin/4module/app/9code/FirstApplication)

Задание 10: Создание приложения для определения местоположения, а также сделать обратное геокодирование (получить адрес по координатам)

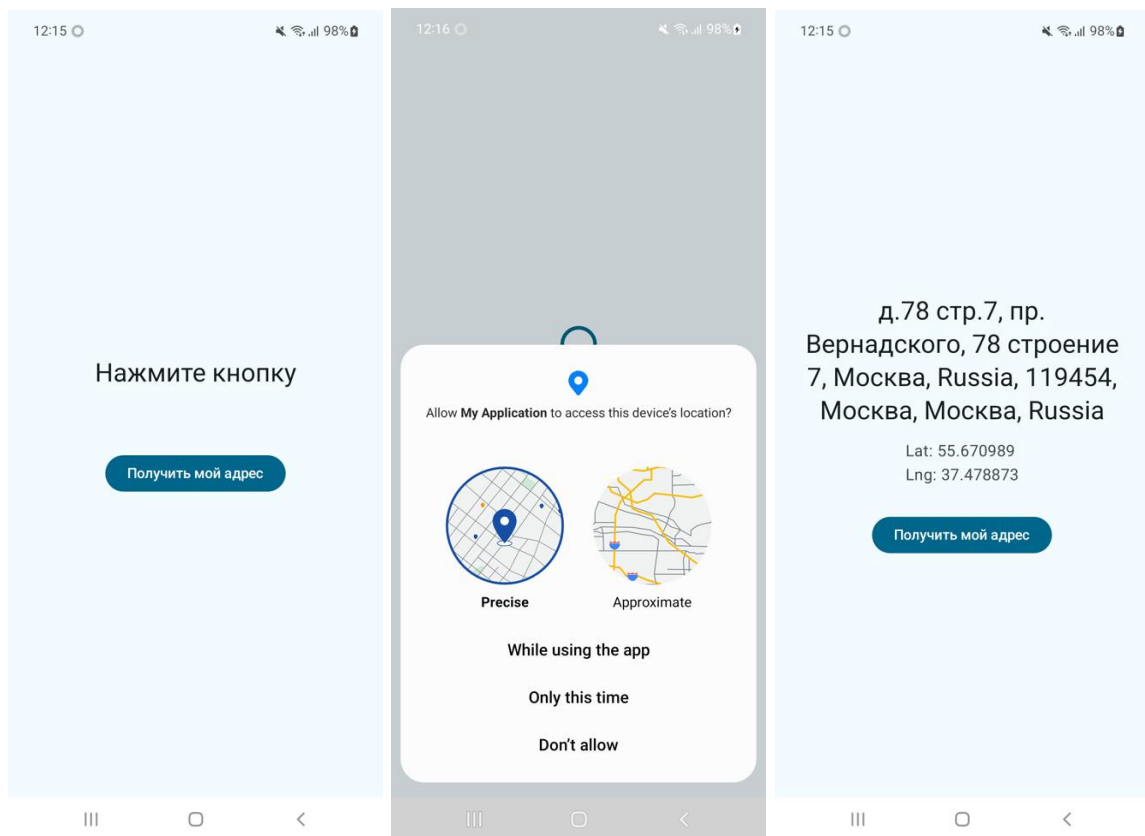
Функционал приложения:

Одна кнопка «Получить мой адрес». После нажатия (и предоставления разрешений):

- Показать индикатор загрузки;
- Получить текущие координаты (latitude, longitude);
- Преобразовать их в читаемый адрес (улица, город, страна и т.д.);
- Вывести адрес крупным текстом;
- Показать краткие координаты (lat / lng).

Требования к реализации:

- Использовать интерфейс: **FusedLocationProviderClient**;
- Обработать разрешения (**ACCESS_COARSE_LOCATION** + **ACCESS_FINE_LOCATION**);
- Можно использовать **Geocoder** для обратного геокодирования;
- Обработать ошибки (нет GPS, нет интернета, пользователь отказал в разрешении и т.д.).



Моя реализация:

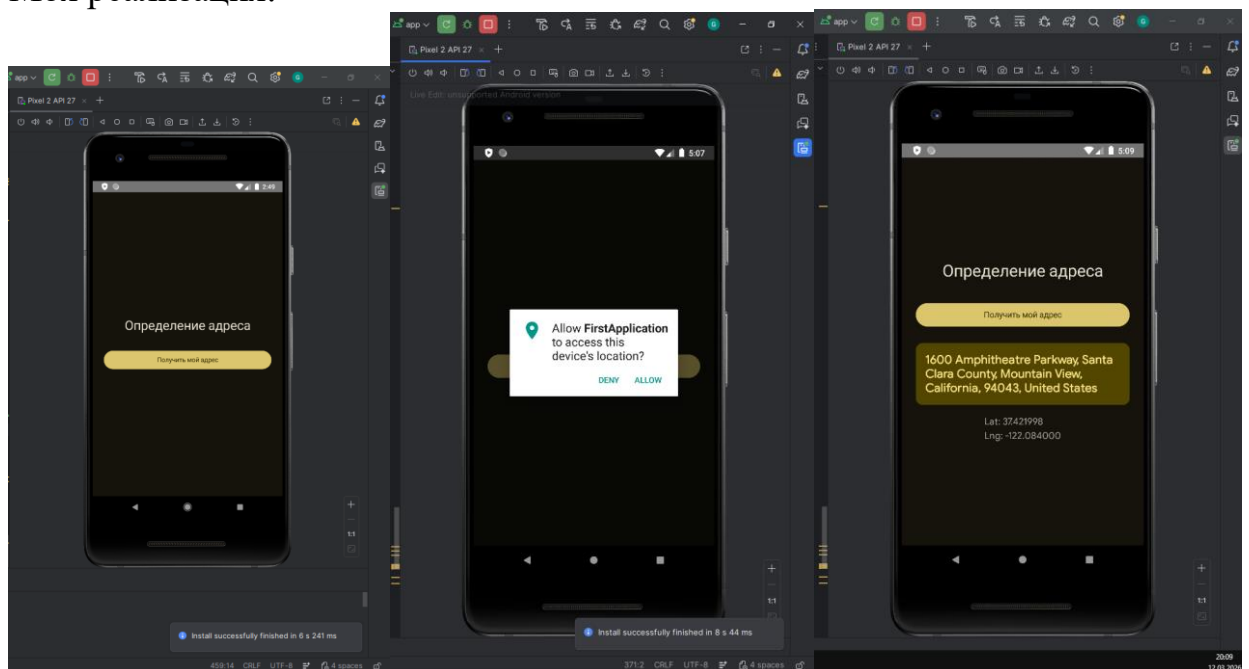


Рисунок 14 – Задание 10

```
package com.example.firstapplication
```

```
import android.Manifest
```

```
import android.annotation.SuppressLint
```

```
import android.content.Context
```

```
import android.content.pm.PackageManager
import android.location.Address
import android.location.Geocoder
import android.location.Location
import android.os.Build
import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.rememberLauncherForActivityResult
import androidx.activity.compose.setContent
import androidx.activity.result.contract.ActivityResultContracts
import androidx.compose.foundation.layout.*
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.core.content.ContextCompat
import com.example.firstapplication.ui.theme.AppTheme
import com.google.android.gms.location.FusedLocationProviderClient
import com.google.android.gms.location.LocationServices
import com.google.android.gms.location.Priority
import com.google.android.gms.tasks.CancellationTokenSource
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.launch
import kotlinx.coroutines.suspendCancellableCoroutine
import kotlinx.coroutines.withContext
import java.io.IOException
import java.util.*
import kotlin.coroutines.resume
import kotlin.coroutines.resumeWithException

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            AppTheme(darkTheme = true) {
                Surface(modifier = Modifier.fillMaxSize()) {
                    LocationScreen()
                }
            }
        }
    }
}
```

```

}

@Composable
fun LocationScreen() {
    val context = LocalContext.current
    val scope = rememberCoroutineScope()

    var isLoading by remember { mutableStateOf(false) }
    var errorMessage by remember { mutableStateOf<String?>(null) }
    var addressText by remember { mutableStateOf<String?>(null) }
    var coordinatesText by remember { mutableStateOf<String?>(null) }

    // Запрос разрешений согласно Android 13+ (POST_NOTIFICATIONS для
    будущих сервисов)
    val permissionLauncher = rememberLauncherForActivityResult(
        contract = ActivityResultContracts.RequestMultiplePermissions()
    ) { permissions ->
        val fineGranted =
permissions[Manifest.permission.ACCESS_FINE_LOCATION] ?: false
        val coarseGranted =
permissions[Manifest.permission.ACCESS_COARSE_LOCATION] ?: false

        if (fineGranted || coarseGranted) {
            scope.launch {
                getLocationAndAddress(
                    context = context,
                    setIsLoading = { isLoading = it },
                    onError = { errorMessage = it },
                    onSuccess = { address, coords ->
                        addressText = address
                        coordinatesText = coords
                    }
                )
            }
        } else {
            errorMessage = "Разрешение на местоположение не предоставлено"
        }

        // Для Android 13+ запрашиваем разрешение на уведомления (на
        будущее)
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
            if (ContextCompat.checkSelfPermission(context,
Manifest.permission.POST_NOTIFICATIONS)
                != PackageManager.PERMISSION_GRANTED) {
                // Можно запросить, но для текущего задания не обязательно
            }
        }
    }
}

```



```

    }
}

fun checkPermissionsAndGetLocation() {
    val fineGranted = ContextCompat.checkSelfPermission(
        context,
        Manifest.permission.ACCESS_FINE_LOCATION
    ) == PackageManager.PERMISSION_GRANTED

    val coarseGranted = ContextCompat.checkSelfPermission(
        context,
        Manifest.permission.ACCESS_COARSE_LOCATION
    ) == PackageManager.PERMISSION_GRANTED

    if (fineGranted || coarseGranted) {
        scope.launch {
            getLocationAndAddress(
                context = context,
                setIsLoading = { isLoading = it },
                onError = { errorMessage = it },
                onSuccess = { address, coords ->
                    addressText = address
                    coordinatesText = coords
                }
            )
        }
    } else {
        // Запрашиваем оба разрешения сразу
        permissionLauncher.launch(
            arrayOf(
                Manifest.permission.ACCESS_FINE_LOCATION,
                Manifest.permission.ACCESS_COARSE_LOCATION
            )
        )
    }
}

Column(
    modifier = Modifier
        .fillMaxSize()
        .padding(24.dp),
    horizontalAlignment = Alignment.CenterHorizontally,
    verticalArrangement = Arrangement.Center,
) {

```

```
Text(
    text = "Определение адреса",
    style = MaterialTheme.typography.headlineMedium
)

Spacer(modifier = Modifier.height(32.dp))

Button(
    onClick = {
        // Очищаем предыдущие результаты
        addressText = null
        coordinatesText = null
        errorMessage = null
        checkPermissionsAndGetLocation()
    },
    enabled = !isLoading,
    modifier = Modifier.fillMaxWidth()
) {
    Text("Получить мой адрес")
}

Spacer(modifier = Modifier.height(24.dp))

if (isLoading) {
    CircularProgressIndicator()
}

if (!errorMessage.isNullOrBlank()) {
    Card(
        colors = CardDefaults.cardColors(
            containerColor = MaterialTheme.colorScheme.errorContainer
        ),
        modifier = Modifier.fillMaxWidth()
    ) {
        Text(
            text = errorMessage!!,
            modifier = Modifier.padding(16.dp),
            color = MaterialTheme.colorScheme.error
        )
    }
}

if (!addressText.isNullOrBlank()) {
    Card(
        modifier = Modifier.fillMaxWidth(),
```

```

        colors = CardDefaults.cardColors(
            containerColor = MaterialTheme.colorScheme.primaryContainer
        )
    ){
        Text(
            text = addressText!!,
            modifier = Modifier.padding(16.dp),
            style = MaterialTheme.typography.bodyLarge.copy(
                fontSize = 20.sp,
                fontWeight = FontWeight.Bold
            )
        )
    }
}

if (!coordinatesText.isNullOrBlank()) {
    Text(
        text = coordinatesText!!,
        style = MaterialTheme.typography.bodyLarge,
        color = MaterialTheme.colorScheme.onSurface.copy(alpha = 0.7f),
        modifier = Modifier.padding(top = 16.dp)
    )
}
}

/**
 * Получение текущего местоположения с использованием
 * FusedLocationProviderClient
 * и современного подхода getCurrentLocation() (рекомендуемый для Android
 * 12+)
 */
@SuppressLint("MissingPermission")
private suspend fun getCurrentLocation(context: Context): Location {
    val fusedLocationClient: FusedLocationProviderClient =
        LocationServices.getFusedLocationProviderClient(context)

    val cancellationTokenSource = CancellationTokenSource()

    return try {
        // Используем современный API getCurrentLocation с приоритетом и
        таймаутом
        fusedLocationClient.getCurrentLocation(
            Priority.PRIORITY_HIGH_ACCURACY, // Высокая точность (GPS +
            Wi-Fi)

```

```

        cancellationTokenSource.token
    ).await()
} catch (e: Exception) {
    throw IOException("Не удалось получить координаты: ${e.message}")
}
}

/**
 * Асинхронное обратное геокодирование с использованием современного
 * GeocodeListener
 * для Android 33+ и обратной совместимостью для старых версий
 */
private suspend fun reverseGeocode(
    context: Context,
    latitude: Double,
    longitude: Double
): Address {
    val geocoder = Geocoder(context, Locale.getDefault())

    return if (Build.VERSION.SDK_INT >=
Build.VERSION_CODES.TIRAMISU) {
        // Для Android 13+ используем асинхронный метод с GeocodeListener
        suspendCancellableCoroutine { continuation ->
            geocoder.getFromLocation(latitude, longitude, 1) { addresses ->
                if (addresses.isNotEmpty()) {
                    continuation.resume(addresses[0])
                } else {
                    continuation.resumeWithException(IOException("Адрес не
найден"))
                }
            }
        }
    } else {
        // Для старых версий используем синхронный метод в фоновом потоке
        withContext(Dispatchers.IO) {
            @Suppress("DEPRECATION")
            val addresses = geocoder.getFromLocation(latitude, longitude, 1)
            if (!addresses.isNullOrEmpty()) {
                addresses[0]
            } else {
                throw IOException("Адрес не найден")
            }
        }
    }
}

```

```

/**
 * Форматирование адреса в читаемый вид
 */
private fun formatAddress(address: Address): String {
    val parts = mutableListOf<String>()

    // Улица и номер дома (приоритет 1)
    val street = address.thoroughfare ?: ""
    val streetNumber = address.subThoroughfare ?: ""
    if (street.isNotEmpty() || streetNumber.isNotEmpty()) {
        parts.add("$streetNumber $street".trim())
    }

    // Район/Округ (приоритет 2)
    address.subAdminArea?.takeIf { it.isNotEmpty() }?.let { parts.add(it) }

    // Город/Населенный пункт (приоритет 3)
    address.locality?.takeIf { it.isNotEmpty() }?.let { parts.add(it) }

    // Область/Регион (приоритет 4)
    address.adminArea?.takeIf { it.isNotEmpty() }?.let { parts.add(it) }

    // Почтовый индекс (приоритет 5)
    address.postalCode?.takeIf { it.isNotEmpty() }?.let { parts.add(it) }

    // Страна (приоритет 6)
    address.countryName?.takeIf { it.isNotEmpty() }?.let { parts.add(it) }

    return if (parts.isNotEmpty()) {
        parts.joinToString(", ")
    } else {
        "Адрес не детализирован"
    }
}

/**
 * Основная функция получения местоположения и адреса
 * С обработкой всех возможных ошибок
 */
private suspend fun getLocationAndAddress(
    context: Context,
    setIsLoading: (Boolean) -> Unit,
    onError: (String) -> Unit,
    onSuccess: (String, String) -> Unit

```

```
) {  
    setIsLoading(true)  
  
    try {  
        // Шаг 1: Получаем координаты через FusedLocationProviderClient  
        val location = try {  
            getCurrentLocation(context)  
        } catch (e: SecurityException) {  
            onError("Нет разрешения на определение местоположения")  
            return  
        } catch (e: IOException) {  
            onError("Ошибка получения координат. Включите GPS.")  
            return  
        } catch (e: Exception) {  
            onError("Неизвестная ошибка при получении координат")  
            return  
        }  
  
        // Форматируем координаты  
        val coords = String.format(  
            "Lat: %.6f\nLng: %.6f",  
            location.latitude,  
            location.longitude  
        )  
  
        // Шаг 2: Получаем адрес через обратное геокодирование  
        val address = try {  
            reverseGeocode(context, location.latitude, location.longitude)  
        } catch (e: IOException) {  
            // Если геокодирование не удалось, показываем только координаты  
            onSuccess("Адрес не определен (проверьте интернет)", coords)  
            return  
        } catch (e: Exception) {  
            onSuccess("Ошибка геокодирования", coords)  
            return  
        }  
  
        // Шаг 3: Форматируем и возвращаем результат  
        val fullAddress = formatAddress(address)  
        onSuccess(fullAddress, coords)  
  
    } finally {  
        setIsLoading(false)  
    }  
}
```

```

/**
 * Extension функция для преобразования Task в корутину
 */
private suspend fun <T> com.google.android.gms.tasks.Task<T>.await(): T {
    return suspendCancellableCoroutine { continuation ->
        addOnCompleteListener { task ->
            if (task.isSuccessful) {
                continuation.resume(task.result!!)
            } else {
                continuation.resumeWithException(task.exception ?:
Exception("Unknown error"))
            }
        }
        addOnCanceledListener {
            continuation.cancel()
        }
    }
}

```

Листинг 17 - MainActivity.kt

```

<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <uses-permission
android:name="android.permission.ACCESS_FINE_LOCATION" />
    <uses-permission
android:name="android.permission.ACCESS_COARSE_LOCATION" />
    <uses-permission
android:name="android.permission.POST_NOTIFICATIONS" />
    <uses-permission
android:name="android.permission.FOREGROUND_SERVICE" />
    <uses-permission
android:name="android.permission.FOREGROUND_SERVICE_DATA_SYNC
" />
    <uses-permission android:name="android.permission.INTERNET" />

    <uses-permission
android:name="android.permission.ACCESS_NETWORK_STATE" />

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"

```

```

    android:icon="@mipmap/ic_launcher"
    android:label="FirstApplication"
    android:roundIcon="@mipmap/ic_launcher_round"
    android:supportsRtl="true"
    android:theme="@style/Theme.FirstApplication"
    tools:targetApi="31">

    <activity
        android:name=".MainActivity"
        android:exported="true"
        android:label="FirstApplication"
        android:theme="@style/Theme.FirstApplication">
        <intent-filter>
            <action android:name="android.intent.action.MAIN" />
            <category android:name="android.intent.category.LAUNCHER" />
        </intent-filter>
    </activity>

    <meta-data
        android:name="com.google.android.geo.API_KEY"
        android:value="YOUR_API_KEY" />

</application>
</manifest>

```

Листинг 18 - AndroidManifest.xml

Задание 11 (AlarmManager): Ежедневное напоминание о таблетке

Функционал приложения:

Пользователь хочет получать уведомление каждый день ровно в 20:00 с текстом «Время принять таблетку!». Приложение должно иметь следующий функционал:

- Позволять включить/выключить напоминание
- Показывать текущее состояние (включено / выключено)
- При нажатии кнопки «Включить» → ставит точный будильник на ближайшие 20:00 (сегодня или завтра)
- При нажатии «Выключить» → отменяет все будильники

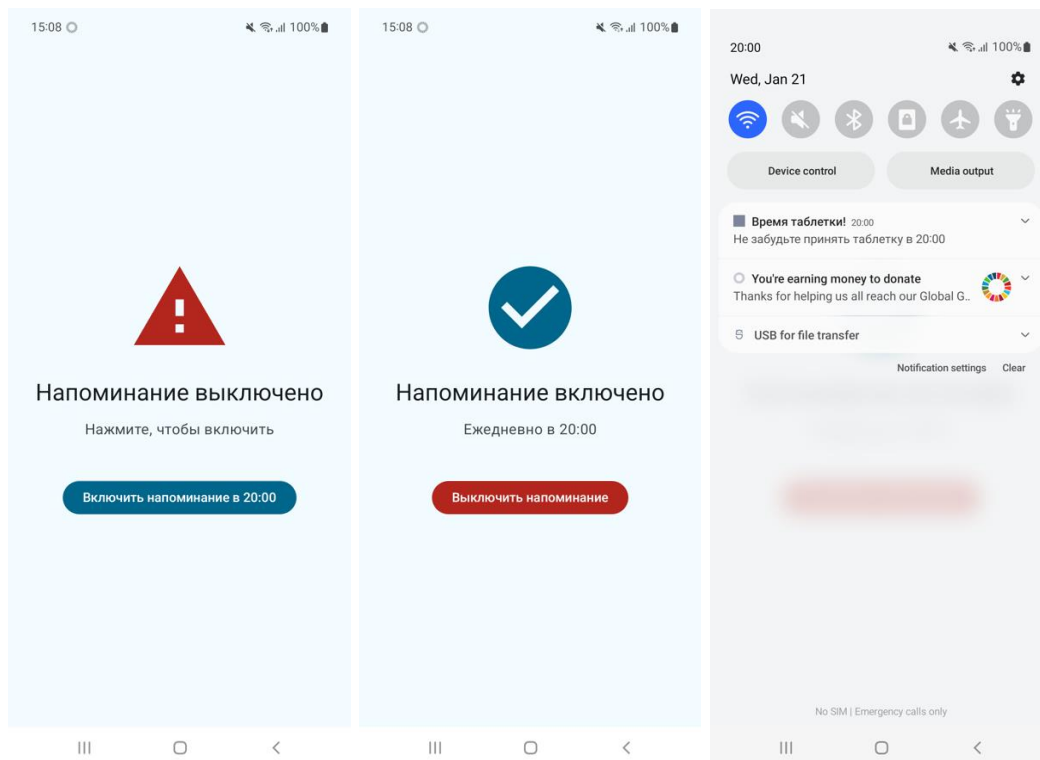
- После перезагрузки телефона напоминание должно восстанавливаться автоматически

Требования к UI:

- Большой текст посередине: «Напоминание о таблетке» + текущее время следующего напоминания;
- Большая кнопка «Включить напоминание» / «Выключить напоминание»;
- Индикатор состояния: зелёный круг + «Включено» / серый + «Выключено»;
- После включения — показывается текст «Следующее напоминание: сегодня/завтра в 20:00»

Технические требования:

- Восстанавливать напоминание после перезагрузки устройства (BOOT_COMPLETED);
- Обработать разрешение POST_NOTIFICATIONS (Android 13+).



Моя реализация:

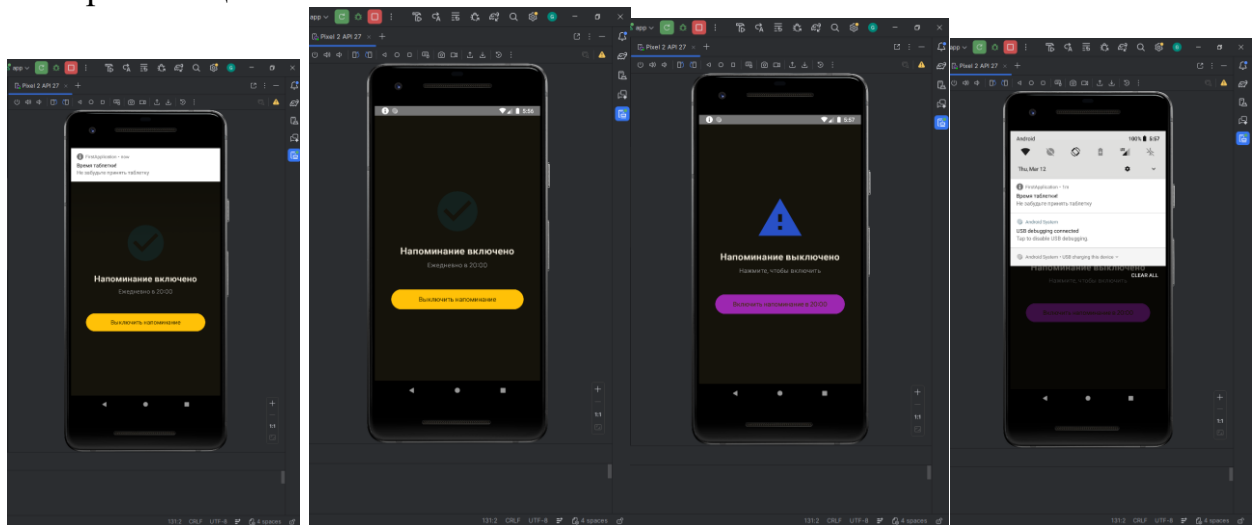


Рисунок 15 – Задание 11

Ссылка на гит:

<https://github.com/oggiorgi/MIREA/tree/main/6sem/kotlin/4module/app/11code/FIRSTApplication>

Задание 12 (cold Flow): Генератор случайных фактов о животных

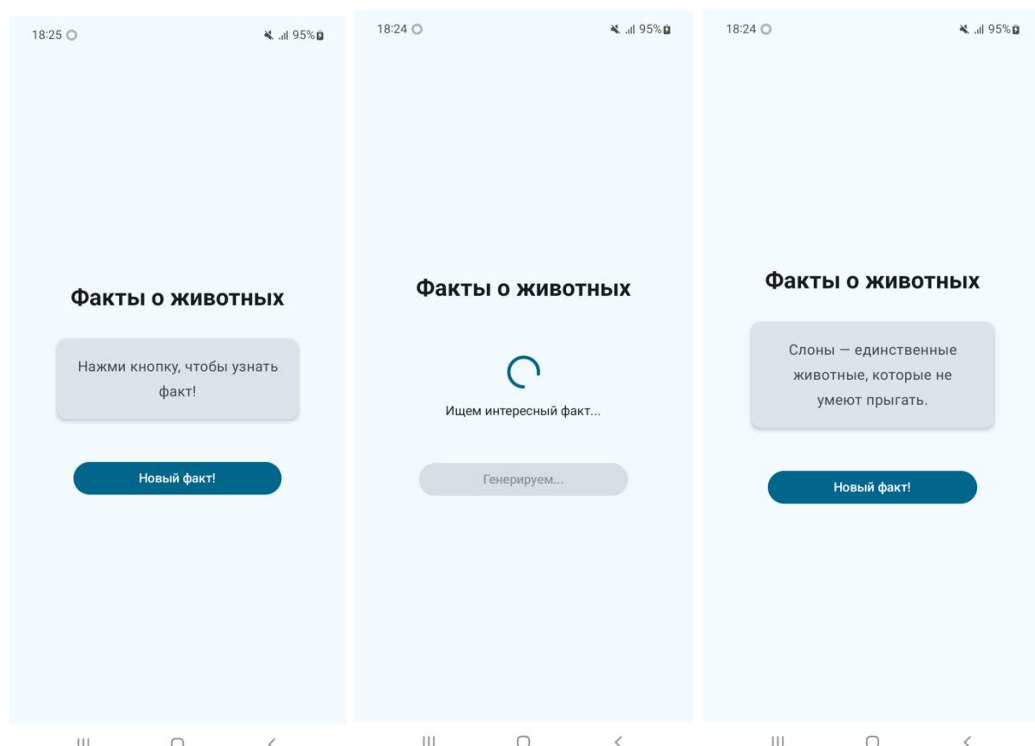
Функционал приложения:

На экране кнопка «Новый факт!». При нажатии генерируется новый случайный факт об животном (из списка или с имитацией задержки). Набор фактов можно взять отсюда: <https://dzen.ru/a/ZKA2BQgB8EUGYNjw> Факт отображается в красивом Card с анимацией появления. Каждый новый collect запускает генерацию заново (cold Flow).

Требования:

- ViewModel возвращает функцию getRandomFact(): Flow<String>;
- Внутри Flow — delay(1500–3000 мс) для имитации загрузки + emit факта;
- UI: кнопка + Card с фактом + индикатор загрузки;
- При повороте экрана факт не генерируется заново (только при нажатии кнопки);
- Минимум 10–15 фактов в списке.

Примерный UI



Моя реализация:

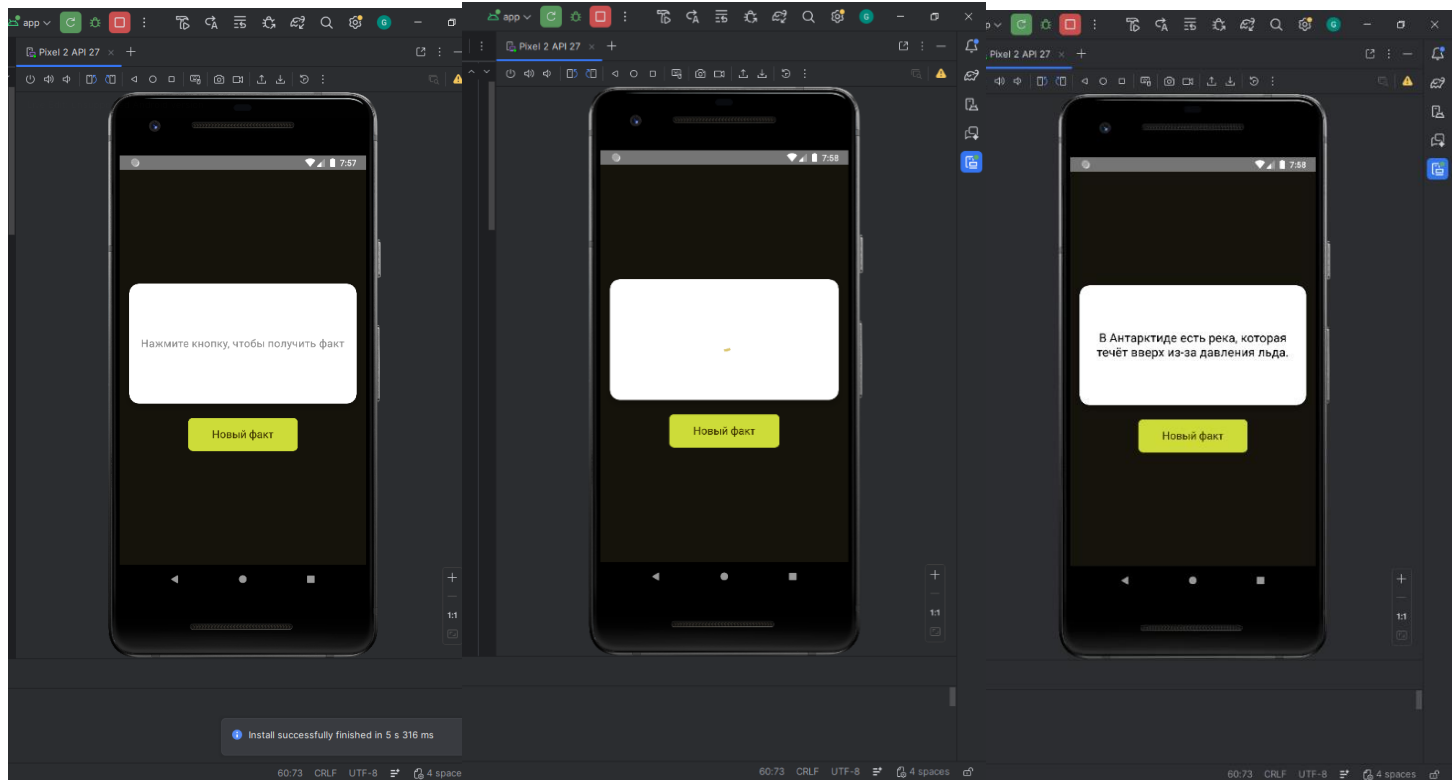


Рисунок 16 – Задание 12

```
package com.example.firstapplication

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.animation.core.Tween
import androidx.compose.animation.animateContentSize
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.runtime.saveable.rememberSaveable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.shadow
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.style.TextAlign
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.example.firstapplication.ui.theme.AppTheme
import kotlinx.coroutines.delay
```

```

import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.flow
import kotlin.random.Random
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            AppTheme(darkTheme = true) {
                Surface(
                    modifier = Modifier.fillMaxSize(),
                    color = MaterialTheme.colorScheme.background
                ) {
                    FactScreen()
                }
            }
        }
    }
}

@Composable
fun FactScreen() {
    var trigger by rememberSaveable { mutableIntStateOf(0) }
    var isLoading by rememberSaveable { mutableStateOf(false) }
    var currentFact by rememberSaveable { mutableStateOf("") }
    val facts = listOf(
        "Водопад Ниагара каждый день переносит около 28 миллионов литров воды.",
        "Человеческое сердце перекачивает около 7 500 литров крови в день.",
        "Самый большой кактус в мире может достигать высоты 20 метров.",
        "Облака не могут двигаться быстрее 30-40 км/ч, иначе они рассеиваются.",
        "Бананы светятся синим под ультрафиолетовым светом.",
        "Самый длинный зафиксированный полёт курицы длился 13 секунд.",
        "Арахис используется для производства динамита.",
        "Клубника — единственный фрукт с семенами снаружи.",
        "В Антарктиде есть река, которая течёт вверх из-за давления льда.",
        "Мед никогда не портится — его находили в гробницах фараонов съедобным.",
        "У отпечатков пальцев коал они почти неотличимы от человеческих.",
        "За всю жизнь человек производит около 40 000 литров слюны.",
        "Глаз страуса больше, чем его мозг.",
        "Огурцы на 96% состоят из воды.",
        "В Японии есть квадратные арбузы, чтобы их было удобно хранить."
    )
    fun getRandomFact(): Flow<String> = flow {
        val delayTime = Random.nextLong(1000, 2000)

```

```

delay(delayTime)
val randomFact = facts.random()
emit(randomFact)
}
LaunchedEffect(trigger) {
    if (trigger > 0) {
        isLoading = true
        currentFact = ""
        getRandomFact().collect { fact ->
            isLoading = false
            currentFact = fact
        }
    }
}
}
Column(
    modifier = Modifier
        .fillMaxSize()
        .padding(16.dp),
    horizontalAlignment = Alignment.CenterHorizontally,
    verticalArrangement = Arrangement.Center
) {
    Card(
        modifier = Modifier
            .fillMaxWidth()
            .shadow(8.dp, RoundedCornerShape(16.dp))
            .animateContentSize(tween(300)),
        shape = RoundedCornerShape(16.dp),
        colors = CardDefaults.cardColors(containerColor = Color.White)
    ) {
        Box(
            modifier = Modifier
                .fillMaxWidth()
                .height(200.dp)
                .padding(16.dp),
            contentAlignment = Alignment.Center
        ) {
            when {
                isLoading -> {
                    CircularProgressIndicator(color =
MaterialTheme.colorScheme.primary)
                }
                currentFact.isNotEmpty() -> {
                    Text(
                        text = currentFact,
                        textAlign = TextAlign.Center,

```

```

        fontSize = 20.sp,
        color = Color.Black,
        modifier = Modifier.padding(8.dp)
    )
}
else -> {
    Text(
        text = "Нажмите кнопку, чтобы получить факт",
        fontSize = 18.sp,
        color = Color.Gray
    )
}
}
}
}
}
Spacer(modifier = Modifier.height(24.dp))
Button(
    onClick = { trigger++ },
    shape = RoundedCornerShape(8.dp),
    colors = ButtonDefaults.buttonColors(containerColor =
Color(0xFFCDDC39))
) {
    Text(
        text = "Новый факт",
        fontSize = 18.sp,
        modifier = Modifier.padding(horizontal = 16.dp, vertical = 8.dp)
    )
}
}
}
}
}

```

Листинг 19 – MainActivity.kt

Задание 13 (StateFlow): Живой курс валют

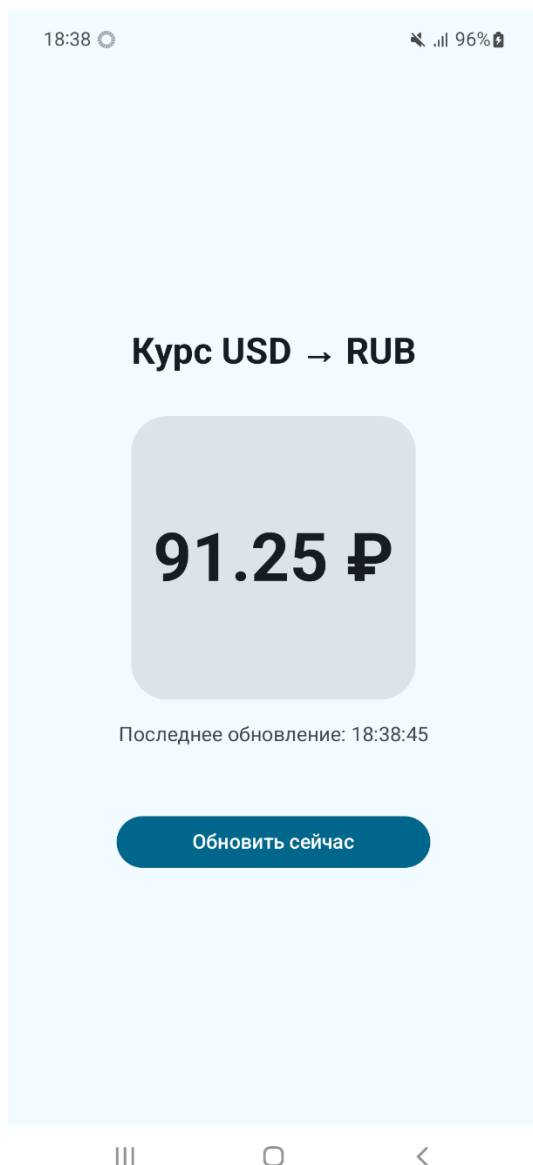
Функционал приложения:

Экран показывает текущий курс доллара к рублю. Курс обновляется каждые 5 секунд (имитация). Есть кнопка «Обновить сейчас» — принудительно запрашивает новый курс.

Требования:

- ViewModel с MutableStateFlow<Double> (курс);
- В init — запускается корутина, которая каждые 5 секунд эммитит случайный курс (например, 90.5 ± 2.0);
- При нажатии кнопки — принудительно генерируется новый курс;
- UI: большой текст с курсом + стрелка вверх/вниз (зелёная/красная) при изменении + кнопка «Обновить»;
- При повороте экрана курс остаётся тем же (StateFlow сохраняет значение).

Примерный UI



Моя реализация:

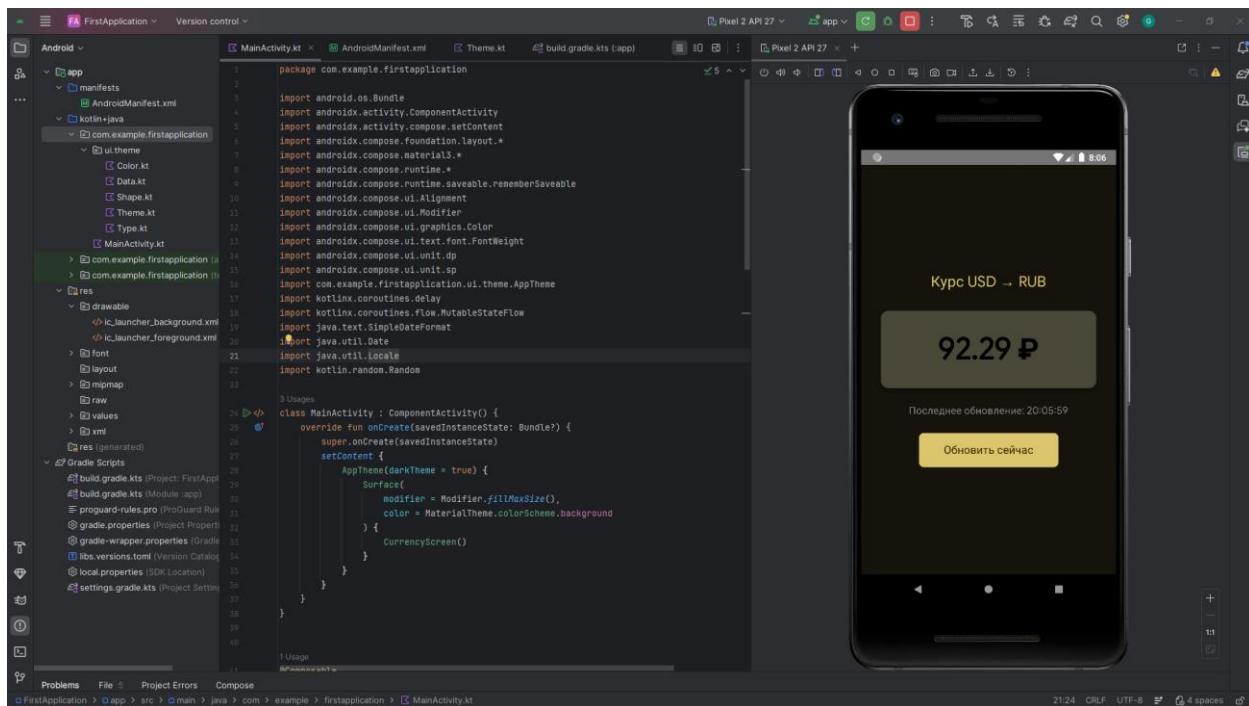


Рисунок 17 – Задание 13

```
package com.example.firstapplication

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.*
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.runtime.saveable.rememberSaveable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.example.firstapplication.ui.theme.AppTheme
import kotlinx.coroutines.delay
import kotlinx.coroutines.flow.MutableStateFlow
import java.text.SimpleDateFormat
import java.util.Date
import java.util.Locale
import kotlin.random.Random

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            AppTheme(darkTheme = true) {
                Surface(
                    modifier = Modifier.fillMaxSize(),
                    color = MaterialTheme.colorScheme.background
                ) {
                    CurrencyScreen()
                }
            }
        }
    }
}
```

```

super.onCreate(savedInstanceState)
setContent {
    AppTheme(darkTheme = true) {
        Surface(
            modifier = Modifier.fillMaxSize(),
            color = MaterialTheme.colorScheme.background
        ) {
            CurrencyScreen()
        }
    }
}
}

@Composable
fun CurrencyScreen() {
    var currentRate by rememberSaveable { mutableDoubleStateOf(90.5) }
    val initialTime = System.currentTimeMillis()
    var lastUpdateTimestamp by rememberSaveable {
mutableLongStateOf(initialTime) }
    val rateStateFlow = remember { MutableStateFlow(currentRate) }.apply {
        value = currentRate
    }

    fun formatTime(timestamp: Long): String {
        val dateFormat = SimpleDateFormat("HH:mm:ss", Locale.getDefault())
        return dateFormat.format(Date(timestamp))
    }

    fun generateRandomRate(): Double {
        return 90.5 + (Random.nextDouble() * 4.0 - 2.0)
    }

    fun updateRate() {
        val newRate = generateRandomRate()
        currentRate = newRate
        lastUpdateTimestamp = System.currentTimeMillis()
        rateStateFlow.value = newRate
    }

    LaunchedEffect(Unit) {
        while (true) {
            delay(5000)
            updateRate()
        }
    }
}

```

```

}
Column(
    modifier = Modifier
        .fillMaxSize()
        .padding(16.dp),
    horizontalAlignment = Alignment.CenterHorizontally,
    verticalArrangement = Arrangement.Center
) {
    Text(
        text = "Куп USD → RUB",
        fontSize = 24.sp,
        fontWeight = FontWeight.Medium,
        color = MaterialTheme.colorScheme.primary,
        modifier = Modifier.padding(bottom = 16.dp)
    )
    Card(
        modifier = Modifier
            .fillMaxWidth()
            .padding(16.dp),
        elevation = CardDefaults.cardElevation(8.dp),
        shape = MaterialTheme.shapes.medium
    ) {
        Box(
            modifier = Modifier
                .fillMaxWidth()
                .padding(32.dp),
            contentAlignment = Alignment.Center
        ) {
            Text(
                text = "%.2f P".format(currentRate),
                fontSize = 48.sp,
                fontWeight = FontWeight.Bold,
                color = Color.Black
            )
        }
    }
    Text(
        text = "Последнее обновление: ${formatTime(lastUpdateTimestamp)}",
        fontSize = 16.sp,
        color = Color.Gray,
        modifier = Modifier.padding(top = 8.dp, bottom = 24.dp)
    )
    Button(
        onClick = { updateRate() },
        shape = MaterialTheme.shapes.small,

```

```

        colors = ButtonDefaults.buttonColors(containerColor =
MaterialTheme.colorScheme.primary)
    ) {
        Text(
            text = "Обновить сейчас",
            fontSize = 18.sp,
            modifier = Modifier.padding(horizontal = 16.dp, vertical = 8.dp)
        )
    }
}
}
}

```

Листинг 20 – MainActivity.kt

Задание 14: Компас

Функционал приложения:

Приложение показывает направление на магнитный север с помощью встроенных сенсоров смартфона. Основные возможности:

1. Отображение азимута

- Показывает текущий азимут (угол от магнитного севера) в градусах (0° — север, 90° — восток, 180° — юг, 270° — запад);
- Значение обновляется в реальном времени при повороте устройства.

2. Анимированная стрелка компаса

- Стрелка плавно поворачивается в сторону севера;
- Красная часть стрелки указывает на север, серая — на юг;
- Поворот анимирован (без резких скачков).

3. Автоматическое управление сенсорами

- Слушатель включается, когда экран виден (onResume);
- Слушатель выключается, когда экран не виден (onPause) — экономия батареи;
- При отсутствии нужного сенсора — показывается сообщение об ошибке.

4. Сохранение поведения при повороте экрана

- Стрелка и значения не «сбрасываются» при изменении ориентации устройства.

Требования к UI:

UI должен быть минималистичным, стильным и удобным для использования в тёмное время суток.

1. Центральный элемент — компас

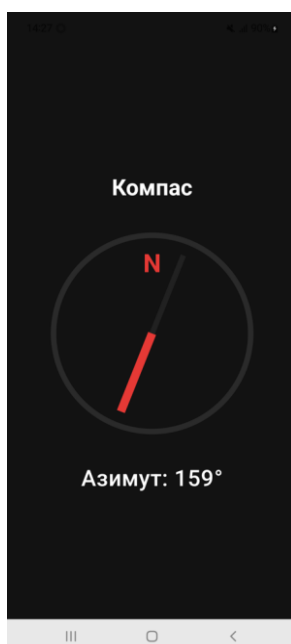
- Круглый диск диаметром ~70–80% ширины экрана;
- Стрелка компаса (двухцветная: красная на север, чёрная на юг);
- Стрелка занимает ~80% диаметра круга;
- Над стрелкой — крупная буква «N»;

2. Текстовая информация

- Вверху экрана — заголовок «Компас»
- Под стрелкой — крупный текст азимута: «Азимут: 142°»;

3. Сообщение об ошибке (если сенсор отсутствует)

- Большой красный текст посередине: «Устройство не поддерживает датчик ориентации».



Моя реализация:

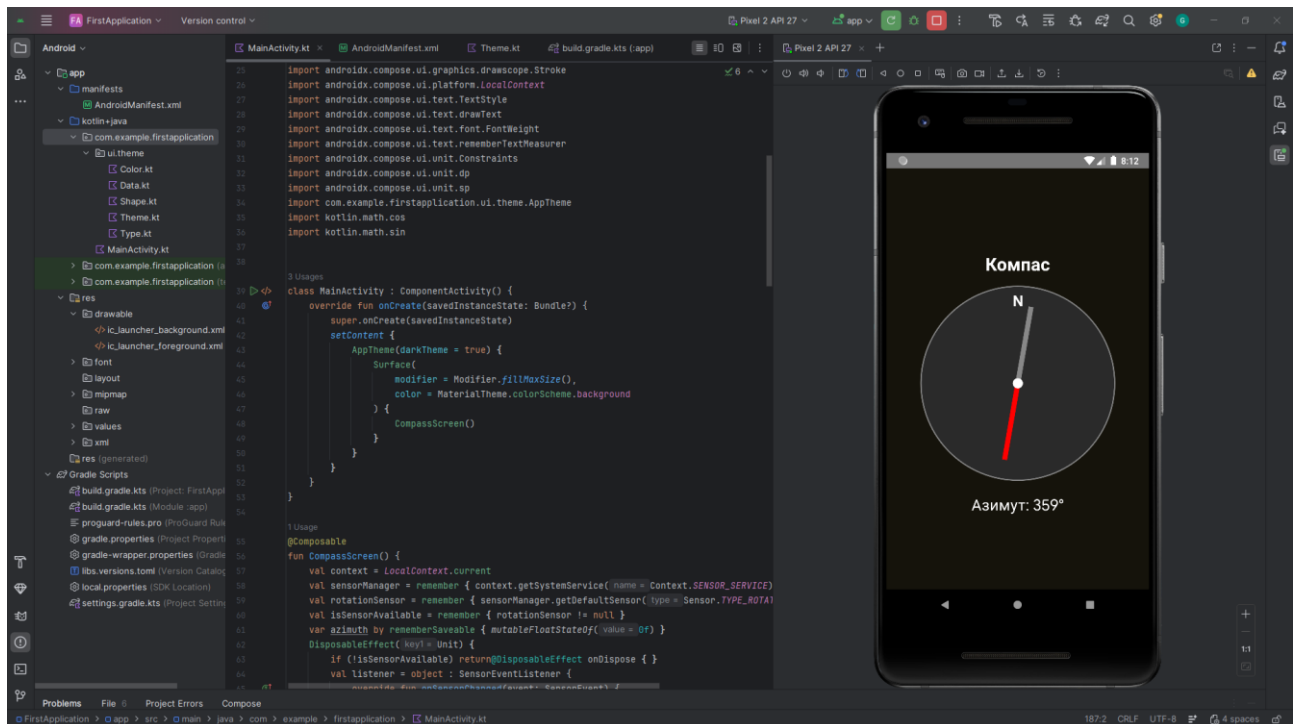


Рисунок 18 – Задание 14

```
package com.example.firstapplication
```

```
import android.content.Context
import android.hardware.Sensor
import android.hardware.SensorEvent
import android.hardware.SensorEventListener
import android.hardware.SensorManager
import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.animation.core.animateFloatAsState
import androidx.compose.animation.core.tween
import androidx.compose.foundation.Canvas
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.runtime.*
import androidx.compose.runtime.saveable.rememberSaveable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.geometry.Offset
import androidx.compose.ui.graphics.Color
```

```

import androidx.compose.ui.graphics.drawscope.Stroke
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.text.TextStyle
import androidx.compose.ui.text.drawText
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.rememberTextMeasurer
import androidx.compose.ui.unit.Constraints
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.example.firstapplication.ui.theme.AppTheme
import kotlin.math.cos
import kotlin.math.sin

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            AppTheme(darkTheme = true) {
                Surface(
                    modifier = Modifier.fillMaxSize(),
                    color = MaterialTheme.colorScheme.background
                ) {
                    CompassScreen()
                }
            }
        }
    }
}

@Composable
fun CompassScreen() {
    val context = LocalContext.current
    val sensorManager = remember {
        context.getSystemService(Context.SENSOR_SERVICE) as SensorManager
    }
    val rotationSensor = remember {
        sensorManager.getDefaultSensor(Sensor.TYPE_ROTATION_VECTOR)
    }
    val isSensorAvailable = remember { rotationSensor != null }
    var azimuth by rememberSaveable { mutableFloatStateOf(0f) }
    DisposableEffect(Unit) {
        if (!isSensorAvailable) return@DisposableEffect onDispose { }
        val listener = object : SensorEventListener {
            override fun onSensorChanged(event: SensorEvent) {
                val rotationMatrix = FloatArray(9)
                SensorManager.getRotationMatrixFromVector(rotationMatrix,

```

```

event.values)
    val orientation = FloatArray(3)
    SensorManager.getOrientation(rotationMatrix, orientation)
    val azimuthRad = orientation[0]
    var azimuthDeg = Math.toDegrees(azimuthRad.toDouble()).toFloat()
    azimuthDeg = (azimuthDeg + 360) % 360
    azimuth = azimuthDeg
}
override fun onAccuracyChanged(sensor: Sensor?, accuracy: Int) {}
}
sensorManager.registerListener(
    listener,
    rotationSensor,
    SensorManager.SENSOR_DELAY_GAME
)
onDispose {
    sensorManager.unregisterListener(listener)
}
}
val animatedAzimuth by animateFloatAsState(
    targetValue = azimuth,
    animationSpec = tween(durationMillis = 200),
    label = "azimuth"
)
Box(
    modifier = Modifier
        .fillMaxSize()
        .padding(16.dp)
) {
    if (!isSensorAvailable) {
        Text(
            text = "Устройство не поддерживает датчик ориентации",
            color = Color.Red,
            fontSize = 18.sp,
            modifier = Modifier.align(Alignment.Center)
        )
    } else {
        Column(
            horizontalAlignment = Alignment.CenterHorizontally,
            verticalArrangement = Arrangement.Center,
            modifier = Modifier.fillMaxSize()
        ) {
            Text(
                text = "Компас",
                fontSize = 28.sp,

```



```

        fontWeight = FontWeight.Bold,
        color = Color.White,
        modifier = Modifier.padding(top = 16.dp)
    )
    Spacer(modifier = Modifier.height(16.dp))
    CompassNeedle(
        azimuth = animatedAzimuth,
        modifier = Modifier
            .fillMaxWidth(0.8f)
            .aspectRatio(1f)
    )
    Spacer(modifier = Modifier.height(24.dp))
    Text(
        text = "Азимут: ${azimuth.toInt()}°",
        fontSize = 24.sp,
        fontWeight = FontWeight.Medium,
        color = Color.White
    )
    }
}
}
}

@Composable
fun CompassNeedle(azimuth: Float, modifier: Modifier = Modifier) {
    val textMeasurer = rememberTextMeasurer()
    val textStyle = TextStyle(
        color = Color.White,
        fontSize = 24.sp,
        fontWeight = FontWeight.Bold
    )
    Canvas(modifier = modifier) {
        val center = Offset(size.width / 2, size.height / 2)
        val radius = size.minDimension / 2
        val needleLength = radius * 0.8f
        val northAngle = Math.toRadians(azimuth.toDouble()).toFloat()
        drawCircle(
            color = Color(0xFF2A2A2A),
            radius = radius
        )
        drawCircle(
            color = Color.Gray,
            radius = radius,
            style = Stroke(width = 2.dp.toPx())
        )
        val northX = center.x + needleLength * sin(northAngle)

```

```

    val northY = center.y - needleLength * cos(northAngle)
    drawLine(
        color = Color.Red,
        start = center,
        end = Offset(northX, northY),
        strokeWidth = 8.dp.toPx()
    )
    val southAngle = northAngle + Math.PI.toFloat()
    val southX = center.x + needleLength * sin(southAngle)
    val southY = center.y - needleLength * cos(southAngle)
    drawLine(
        color = Color.Gray,
        start = center,
        end = Offset(southX, southY),
        strokeWidth = 8.dp.toPx()
    )
    drawCircle(
        color = Color.White,
        radius = 8.dp.toPx(),
        center = center
    )
    val textLayoutResult = textMeasurer.measure(
        text = "N",
        style = textStyle,
        constraints = Constraints()
    )
    val textX = center.x - textLayoutResult.size.width / 2
    val textY = center.y - radius * 0.85f - textLayoutResult.size.height / 2
    drawText(textLayoutResult, topLeft = Offset(textX, textY))
}
}

```

Листинг 21 – MainActivity.kt

Выводы

В ходе выполнения практических работ модуля были получены комплексные навыки разработки Android-приложений, охватывающие современные подходы к асинхронной обработке, управлению фоновыми задачами и взаимодействию с аппаратным обеспечением устройства. Освоены принципы работы с корутинами для параллельного выполнения операций, организации отменяемых задач и обработки потоков данных, что позволило эффективно управлять фоновыми операциями без блокировки основного потока пользовательского интерфейса, что критически важно для создания отзывчивых приложений.

Получены знания о системных компонентах Android, включая сервисы различных типов, где особое внимание уделено Foreground Services с обязательным уведомлением и указанием типа для современных версий Android. Освоены методы запуска, остановки и взаимодействия с сервисами через Intent и ServiceConnection. Получен практический опыт использования WorkManager для гарантированного выполнения отложенных и периодических работ с учетом жизненного цикла приложения, включая реализацию цепочек задач с передачей данных между Worker'ами и отслеживанием состояния через LiveData. Изучена работа с BroadcastReceiver для реакции на системные события, такие как завершение загрузки устройства и срабатывание точных будильников через AlarmManager, что позволило реализовать автоматическое восстановление напоминаний после перезагрузки устройства.

Получен практический опыт создания уведомлений различных типов с использованием NotificationCompat и каналов уведомлений для современных версий Android. Освоена работа с runtime-разрешениями, включая запрос разрешений на точное и приблизительное местоположение, обработку разрешения на уведомления и запрос разрешения на точные будильники через системные настройки.

Освоена работа с геолокацией через современный API FusedLocationProviderClient, который интеллектуально комбинирует данные от GPS, Wi-Fi и сотовых вышек. Реализовано обратное геокодирование с использованием Geocoder, включая обработку ошибок при отсутствии интернет-соединения или недоступности сервиса. Применены различные приоритеты точности в зависимости от сценария использования.

В ходе работ активно использовался современный инструментарий Jetpack Compose для построения декларативного UI, где реализованы экраны с различными компонентами: кнопки управления, индикаторы загрузки, карточки для отображения результатов, кастомизация тем и цветовых схем. Освоено управление состоянием через mutableStateOf и remember, а также интеграция с LiveData для реактивного обновления интерфейса при изменении данных от WorkManager.

Особое внимание уделено корректной обработке исключительных ситуаций, включая отсутствие необходимых разрешений, отсутствие GPS-сигнала или интернет-соединения, таймауты при геокодировании, ситуации когда пользователь навсегда запретил разрешение, а также ограничения Android разных версий. В процессе выполнения работ применялись лучшие практики Android-разработки, включая разделение ответственности между компонентами, использование синглтонов для менеджеров и хелперов, работу с SharedPreferences для хранения простых состояний, использование PendingIntent с соответствующими флагами безопасности и проверку возможностей системы перед выполнением операций.